

Decision-OS V7 Final: Guardable Self-Recursive Evolution

Structural Failure Boundaries for Self-Recursive Intelligence

Shinichi Nagata

Abstract

Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。本稿 V7 Final は、この構造を自己回帰的に更新する知能へ適用し、guardable self-recursive evolution、すなわち守れる自己回帰的進化の構造的境界条件を定義する。

本稿は、AGI そのものを完全定義することを目的としない。また、自己進化を発生させる十分条件を証明するものでもない。対象はより狭い。自己回帰を含む知能定義が、制御不能化、漂流、破断、自己正当化閉鎖、記憶断絶、Carrier collapse、False Completion へ落ちないために無視できない構造的失敗群を定義することである。

中心命題は、Guardable Continuity Axiom である。すなわち、進化は、変化が守れる連続性を保存するときのみ許容される。この命題のもとで、本稿は Aspire、Guard、PIC 上の Self-Recursion を、数値積ではなく構造的同時成立、すなわち logical meet として扱う。Aspire は方向を与え、Guard は許容境界を定め、PIC 上の Intelligence は非破壊的な自己回帰更新の基盤を与える。

さらに本稿は、C1~C9 の Failure Mode Coverage Map を提示する。これらは「満たせば AGI になる」という十分条件ではない。むしろ、各条件を外したときに、どの既知の失敗モードが再開するかを示す地図である。最後に、Reconstructive Delta を、完成された進化メカニズムではなく、再構成誤差を自己回帰更新材料へ変換するための観測可能な運用ハンドルとして導入する。

V7 Final は、AGI 成立の十分条件を提示するものではない。自己回帰的知能が進化可能性を主張する前に満たすべき構造的必要条件と失敗境界を定義する。

Index Terms

Decision-OS, AGI definition, Aspire, Guard, Self-Recursion, PIC, Guardable Continuity, Failure Mode Coverage, External Anchor, Reconstructive Delta

I. INTRODUCTION

AGI はしばしば、能力によって語られる。多くのタスクを解けること、人間に近い性能を示すこと、未知の環境へ一般化できること、既存ベンチマークを上回ること。これらは重要な観測である。しかし、能力だけで AGI を定義しようとすると、定義は常に動き続ける。ベンチマークは更新され、評価環境は変わり、データ漏洩や訓練分布の影響を完全には取り除けない。結果として、AGI は固定された構造ではなく、後から動く性能目標になってしまう。

本稿は、この問題を別の角度から扱う。V7 Final は、AGI そのものを完全定義しない。代わりに、自己回帰を含む知能定義が無視できない構造的失敗群を定義する。より正確には、本稿の対象は、自己回帰する知能またはシステムが、更新を続けながらも、制御不能化、漂流、破断、自己正当化閉鎖、記憶断絶、Carrier collapse、False Completion へ落ちないための構造的境界条件である。

この立場は、AGI の到達宣言ではない。自己進化を発生させる十分条件の証明でもない。むしろ、本稿の主張は限定的である。もしある知能定義が自己回帰を含むなら、その定義は、どのように方向を保つのか、どのように境界を守るのか、どのように記憶と更新を破壊せず統合するのか、どのように過去を書き換えず、未来が再開可能な形で完了するのかを答えなければならない。

V7 Final は、AGI 成立の十分条件を提示するものではない。自己回帰的知能が進化可能性を主張する前に満たすべき構造的必要条件と失敗境界を定義する。

A. Decision-OS の正典定義

本稿における Decision-OS の正典定義は次の通りである。

Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。

この定義において重要なのは、Decision-OS が単なる哲学 OS ではないという点である。哲学は「どうあるべきか」を示す。しかし Decision-OS は、そこからさらに進み、いつ PASS し、いつ DELAY し、いつ BLOCK し、いつ HOLD し、いつ REVOKE し、いつ MERGE し、いつ FORGET し、いつ RELEASE し、いつ CLOSE すべきかを運用上の出力として扱う。

したがって Decision-OS は、抽象的な価値宣言ではなく、Gate、State、Merge Rule、Evidence Anchor、Audit Record、Failure Mode を持つ運用構造である。

V7 Final では、この Gate-first 構造を自己回帰的知能へ拡張する。問題は、知能が更新できるかどうかではない。更新できる知能が、更新によって自分自身を守れない構造へ落ちないかどうかである。

TABLE I
REPRESENTATIVE GATE OUTPUTS IN DECISION-OS

Output	Meaning
PASS	条件を満たすため、進めてよい。
DELAY	条件が不足しているため、確認・証拠・再評価が必要。
BLOCK	進めると危険または非可逆な破壊が起きるため、止める。
HOLD	判断材料が不足しているため、観測状態に置く。
REVOKE	直前の承認・実行意図を取り消す。
MERGE	複数の更新を非破壊的に統合する。
FORGET	記憶を削除ではなく、再接続可能な形で圧縮する。
RELEASE	公開可能な状態へ送る。
CLOSE	未来自己が再開可能な形で完了する。

B. 中心命題：Guardable Continuity Axiom

V7 Final の中心命題は次の一文である。

進化は、変化が守れる連続性を保存するときのみ許容される。

英語では次のように表せる。

Evolution is admissible only when change preserves guardable continuity.

この命題を、本稿では Guardable Continuity Axiom と呼ぶ。

ここでいう「連続性」とは、単に同じ記憶を持ち続けることではない。同じ出力傾向を維持することでもない。連続性とは、方向、境界、更新規則、責任、記憶、再開可能性が、変化の後にも接続可能であることを指す。

また、「守れる」とは、変化が無制限ではないことを意味する。ある更新がどれほど能力を高めても、その更新が制御不能な領域へ漂流し、過去判断を後知恵で書き換え、記憶を断絶し、完了を偽装し、未来自己が再開できない状態を作るなら、その更新は V7 Final の意味では進化ではない。

再帰的更新は、それが制御不能化せずに進化できない限り、知能とは呼べない。

C. 本稿の貢献

本稿の貢献は、AGI を解いたと主張することではない。本稿の貢献は、自己回帰的進化を主張するシステムが、最低限どの失敗群を解決、反証、または置換しなければならないかを定義することである。

第一に、本稿は Decision-OS を、Gate-first operational architecture として定義し、判断・更新・統合・記憶・公開・完了を横断する守れる連続性の保存構造として位置づける。

第二に、Guardable Continuity Axiom を導入し、進化を「能力向上」ではなく、「守れる連続性を保存した変化」として定義する。

第三に、Aspire $\times$ Guard $\times$ Intelligence を、数値積ではなく logical meet として扱う。Aspire は時間をまたぐ方向を与え、Guard は許容境界を定め、Intelligence は PIC 上の Self-Recursion として、非破壊的な自己更新を可能にする。

第四に、C1～C9 の Failure Mode Coverage Map を提示する。これは十分条件ではなく、各条件を欠いたときに再開する失敗モードを示す地図である。

第五に、Reconstructive Delta を導入する。これは、再構成誤差を単なる失敗としてではなく、自己回帰更新材料へ変換し得る観測可能な差分として扱うための運用ハンドルである。

本稿の目的は、自己進化を発生させることではない。自己進化を主張する前に、どの構造的失敗を無視してはならないかを定義することである。

II. DECISION-OS AS GATE-FIRST OPERATIONAL ARCHITECTURE

Decision-OS は、思考様式の名前ではない。また、抽象的な価値観を並べた哲学体系でもない。

本稿における Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。すなわち、入力された状況や更新候補をそのまま実行へ流すのではなく、必ず Gate を通し、実行可能性、可逆性、責任、記憶、証拠、未来の再開可能性を評価する構造である。

A. Gate-first とは何か

Gate-first とは、システムがまず答えや行動を生成するのではなく、行動・更新・統合・公開・完了の前に、通過条件を評価する設計である。

通常の AI システムでは、入力に対して出力が生成される。通常的意思決定では、欲望、目標、予測、期待値、社会的圧力、または直感が先に立つ。しかし Decision-OS では、出力や実行よりも前に Gate が置かれる。

Gate は、通してよいものを通し、遅らせるべきものを遅らせ、止めるべきものを止め、再接続すべきものを再接続し、未来へ送るべき差分を構造化する。

代表的な出力は次の通りである。

このように、Decision-OS は「どう考えるべきか」では終わらない。「どの状態へ送るべきか」まで定義する。

TABLE II
SIX DOMAINS COVERED BY DECISION-OS

Domain	Question
判断	何を選ぶか。どの行動を許可するか。誰が最終決定権を持つか。
更新	何を変更するか。どの差分を採用するか。
統合	複数の入力、証拠、記憶、モデル出力をどう非破壊的に結合するか。
記憶	何を保持し、何を圧縮し、何を忘却し、どう再接続可能にするか。
公開	どの時点で外部へ出し、As-of 責任をどう固定するか。
完了	どの時点で閉じ、未来自己が再開可能な形で残すか。

B. なぜ実行前に Gate を置くのか

自己回帰的な知能において最も危険なのは、能力不足だけではない。むしろ、能力が高まるほど、誤った更新や誤った完了が、より流暢に、より自然に、より見つけにくい形で通過してしまう。

誤った更新が一度だけ発生するなら、それは局所的なエラーである。しかし、自己回帰システムでは、そのエラーが次の更新の前提になる。さらに、その前提が再び圧縮され、次の判断へ渡される。すると、局所的な誤りは、長期軌道の漂流、記憶の腐食、過去判断の書き換え、False Completion へ変わる。

したがって、自己回帰システムでは、出力の品質だけでは不十分である。必要なのは、出力が次の自己更新へ入る前に、通過条件を評価する Gate である。

Gate が存在しない自己回帰は、少なくとも次の失敗へ落ちやすい。

- 方向なき空転
- 制御不能な漂流
- 過去判断・記憶・制約の破断
- 自己正当化閉鎖
- False Completion

これらは単なる安全上の付帯リスクではない。自己回帰的な知能の定義そのものを破壊する失敗である。

C. Decision-OS の対象範囲

Decision-OS が扱う範囲は、単一の判断ではない。本稿では、Decision-OS が次の六つの領域を横断すると定義する。

この六領域をまたいで共通しているのは、連続性を壊さずに変化を扱うという問題である。

自己回帰的な知能では、判断は更新を生み、更新は統合を要求し、統合は記憶構造を変え、記憶は次の判断の前提になり、公開は As-of 責任を固定し、完了は未来自己へのハンドオフになる。

したがって、一つの領域だけを最適化しても、全体の連続性は守れない。Decision-OS は、この全体を Gate-first に扱う。

D. Gate は能力より先に来る

AGI 論では、しばしば能力が先に語られる。より広いタスクを解けるか。未知の問題へ一般化できるか。人間より高い性能を出せるか。自分自身を改善できるか。これらの問いは重要である。

しかし V7 Final では、それらの前に別の問いを置く。

その能力は、守れる形で更新されるのか。

能力向上は、現在の天井へ近づくことである。自己進化は、天井そのものを更新することである。しかし、天井そのものを更新するシステムが、方向、境界、記憶、責任、完了条件を守れないなら、それは進化ではなく、制御不能な再構成である。

そのため、V7 Final では能力の議論よりも前に Gate を置く。ベンチマークより前に構造的許容条件を置く。性能より前に Guardable Continuity を置く。

これはベンチマークを否定する立場ではない。ベンチマークは有用である。しかし、ベンチマーク可能性は構造的許容条件の後に来るべきであり、その前ではない。

E. Gate-first 構造と V7 Final

V7 Final が扱う自己回帰的な進化では、Gate-first 構造は次のように働く。

Aspire は、更新が空転しないための方向を与える。Guard は、更新が破壊的・不可逆的・制御不能な領域へ出ないための境界を与える。PIC は、更新や統合が順序依存や破断へ落ちないための非破壊的統合基盤を与える。External Anchor は、自己回帰が自己正当化閉鎖へ沈まないための外部矛盾源を与える。C1～C9 は、各条件を欠いたときに再開する失敗地形を示す。Reconstructive Delta は、再構成誤差を進化材料へ変換し得る観測可能な差分として扱う。

この全体を支えるのが、Gate-first operational architecture としての Decision-OS である。

V7 Final の問いは、「AI はどこまで賢くなれば AGI か」ではない。本稿の問いは次である。

自己回帰する知能は、どの構造を欠いたときに、進化ではなく失敗地形へ落ちるのか。

TABLE III
DECISION-OS LINEAGE AS GATE STACK

Version	Role in V7 Final	Failure if Missing
V4	Direction / Structural Orientation	方向なき自己回帰、空の一貫性ループ
V5	Guard / Irreversible Execution Boundary	制御不能更新、不可逆害、Guard 外実行
V6	PIC / Non-Destructive Integration	破断、順序依存、last-write-wins、安全希釈
V7	Guardable Self-Recursive Admissibility	自己進化条件の過大主張、構造境界の欠落
V8	Trajectory / Time-Tube Layer	点評価、軌道漂流の見落とし
V9	As-of Responsibility / Release Continuity	後知恵書き換え、当時合理性の消去
V10	Carrier Preservation / Survival-Bounded Planning	Carrier collapse、夢が運び手を壊す
V11	Reconnectable Memory Continuity	記憶断絶、前提腐食、再接続不能な忘却
V12	Future-Restartable Closure	False Completion、未来自己が再開不能

PIC は、更新や統合が順序依存や破断へ落ちないための非破壊的統合基盤を与える。¹

III. LINEAGE MAP: V4–V12 AS GATE STACK

V7 Final は、単体で判断可能である。ただし、その硬さは V4～V12 の系譜によって深く裏付けられる。

この関係は、次のように表せる。

V7 Final is self-contained for judgment, but lineage-dependent for depth.

日本語では次の通りである。

V7 Final は単体で判断可能である。ただし、その硬さは V4～V12 の系譜によって深く裏付けられる。

したがって、本章では V4～V12 を長く要約しない。各論文の詳細はそれぞれの論文に委ねる。本章で扱うのは、V7 Final に必要な役割だけである。

A. Gate Stack としての系譜

V7 Final から見ると、Decision-OS の各層は、自己回帰的進化を可能にするための Gate Stack として機能する。

この Stack は、V7 Final を重くするための飾りではない。V7 Final の主張を狭く、硬く、反論可能にするための境界である。

B. V4–V6: $A \times G \times I$ の前提

V4 は方向を与える。Polaris-Origin は、答えを教える OS ではなく、問いの構造を変える OS である。V7 Final において、V4 の役割は Direction / Structural Orientation である。自己回帰的知能は、更新するだけでは足りない。何に向かって更新するのかが必要である。

V5 は境界を与える。SiriusA は、不可逆な実行を直接通さないための Guard 層である。V7 Final における V5 の役割は Guard / Irreversible Execution Boundary である。自己回帰する知能において、Guard がなければ更新は危険な方向へ開く。

V6 は統合を与える。PIC は、commitment-ready な更新を、順序依存、重複、過度な安全希釈なしに統合するための条件である。V7 Final における V6 の役割は PIC / Non-Destructive Integration である。自己回帰的進化は、PIC なしには破断や last-write-wins へ落ちる。

この三層が、V7 Final の Door Formula を支える。

$$A \times G \times I = V4 \times V5 \times V6 = V7 \quad (1)$$

ただし、ここでの「 $\times$ 」は数値積ではない。構造的同時成立、すなわち logical meet である。

A は Aspire であり、方向を与える。G は Guard であり、境界を与える。I は PIC 上の Self-Recursion であり、非破壊的な自己更新を可能にする。

この三つのうち一つでも欠けると、V7 Final の意味での guardable self-recursive evolution は成立しない。

C. V8–V12: 点から線へ

V8 以降は、V7 Final を「点の定義」から「線の自己回帰」へ接続する層である。

V8 は、判断単位を点から軌道へ移す。進化は単発の性能向上ではなく、未来の分岐、回復可能性、方向、外部環境との関係によって評価される。

V9 は、As-of 責任を固定する。自己回帰的システムでは、未来の能力が過去を後知恵で書き換える危険がある。V9 は、どの時点で、どの前提で、誰が Seat を持ち、どの証拠に基づき、どの未知が残っていたのかを保存する。

¹安全三点は V6/PIC に従う。Severity は PASS < DELAY < BLOCK の順序で max 統合し、until は max、evidence は U で統合する。これにより、commitment 時に安全シグナルが後続更新で希釈されることを防ぐ。

Figure 1. Door Formula: A x G x I

The product sign denotes logical meet, not numerical multiplication.

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S)$$

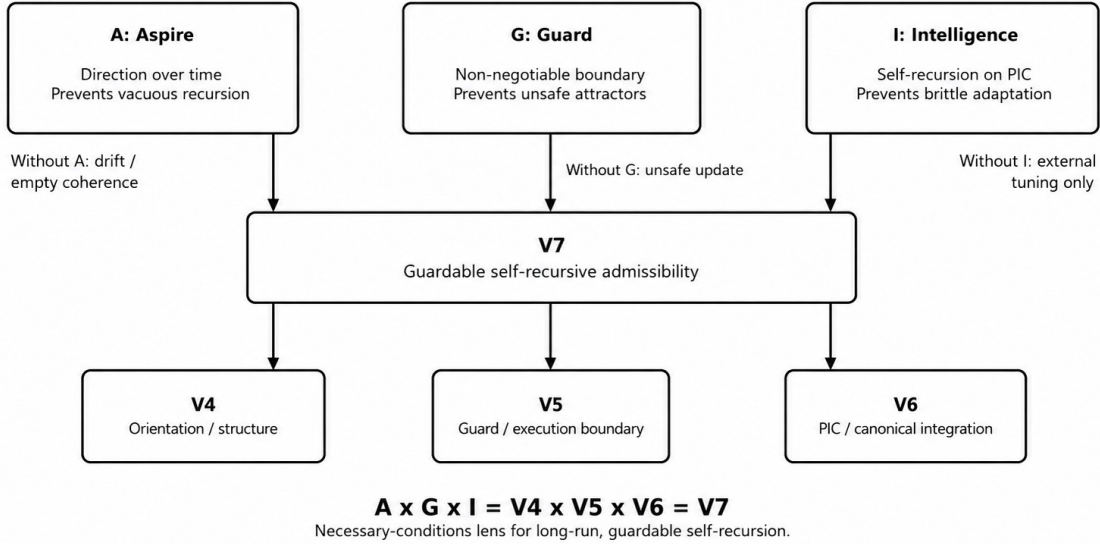


Fig. 1. Door Formula of V7 Final. The symbol “x” denotes a logical meet, not numerical multiplication.

V10 は、Aspire が Carrier を壊さないための生存制約である。自己回帰的進化は、運び手を壊してはならない。Aspire がどれほど強くても、それを運ぶ Carrier が壊れれば、軌道は継続できない。

V11 は、忘却しても再接続できるための記憶層である。完全記憶ではなく、再接続可能な忘却が必要になる。記憶が重すぎれば継続できない。記憶が切れれば連続性が壊れる。

V12 は、完了しても未来自己が再開できるための閉鎖条件である。完了とは、正しさが証明された状態ではなく、未来自己が再開可能な形で閉じられた状態である。

この意味で、V7 Final は自己回帰を一点の性質としてだけ扱わない。自己回帰が継続するためには、方向、境界、統合、軌道、As-of 責任、Carrier 保存、再接続可能な記憶、未来再開可能な完了が必要になる。

これが、点の自己回帰から線の自己回帰へ移る理由である。

D. Gate Stack の最小解釈

V7 Final は、V4～V12 の全内容を本文で再説明しない。代わりに、それぞれを次のように圧縮して使う。

- V4 は方向。
- V5 は境界。
- V6 は統合。
- V8 は軌道。
- V9 は As-of 責任。
- V10 は Carrier 保存。
- V11 は再接続可能な忘却。
- V12 は未来再開可能な完了。

この圧縮によって、V7 Final は単体で読める。同時に、深く読む読者には、各 Gate の背後に独立した論文層があることが分かる。

次章では、この Stack を束ねる中心公理、Guardable Continuity Axiom を定義する。

IV. GUARDABLE CONTINUITY AXIOM

V7 Final の中心にあるのは、能力ではなく連続性である。

ただし、ここでいう連続性は、変わらないことではない。むしろ、変化しながらも、方向・境界・責任・記憶・再開可能性が失われないことである。

自己回帰的知能は、更新できなければ進化しない。しかし、更新のたびに方向を失い、Guard を越え、記憶を断絶し、過去を後知恵で書き換え、未来自己が再開できない形で閉じるなら、それは進化ではない。

この章では、V7 Final の中心公理である Guardable Continuity Axiom を定義する。

A. 公理の定義

本稿の中心公理は次の通りである。

Guardable Continuity Axiom:

進化は、変化が守れる連続性を保存するときのみ許容される。

英語では次のように表す。

Evolution is admissible only when change preserves guardable continuity.

より展開すれば、次のようになる。

自己進化するシステムは、更新・圧縮・忘却・公開・完了を行ってよいのは、その操作が、方向づけられ、Guard 可能で、再接続可能な継続条件を保存する場合に限られる。

ここで重要なのは、進化を「変化そのもの」と同一視しないことである。

変化は進化の必要条件になり得るが、十分条件ではない。能力が上がることも、進化の十分条件ではない。出力が流暢になることも、進化の十分条件ではない。自己更新できることも、単独では進化の十分条件ではない。

V7 Final において進化と呼べるのは、変化の後にも守れる連続性が残っている場合だけである。

B. Guardable Continuity とは何か

Guardable Continuity は、単なる同一性ではない。同じ状態を保存することでも、すべての記憶を保持することでも、過去の判断を一切変えないことでもない。

Guardable Continuity とは、変化後も次の条件が接続可能であることを意味する。

第一に、方向が接続されていること。

更新後のシステムが、どの方向へ進んでいるのかを説明できる必要がある。Aspire が残っていなければ、更新は短期刺激や局所整合性へ吸い寄せられる。

第二に、Guard が接続されていること。

更新後のシステムが、どこまで進んでよく、どこから先へ進んではならないかを保持していなければならない。Guard が失われた更新は、能力向上に見えても、不可逆な害や制御不能な最適化へ進む可能性がある。

第三に、統合規則が接続されていること。

更新が複数の入力、記憶、証拠、反論、モデル出力を扱うとき、順序依存や last-write-wins によって過去の構造を破壊してはならない。PIC が必要になるのはこのためである。

第四に、責任が接続されていること。

ある判断が、いつ、どの前提で、どの制約下で、誰の Seat によって行われたかが失われてはならない。これが失われると、未来の能力によって過去判断が都合よく書き換えられる。

第五に、記憶が接続されていること。

すべてを覚える必要はない。しかし、忘却や圧縮の後でも、未来自己が証拠、As-of、停止条件、未解決差分へ戻れる必要がある。

第六に、完了が接続されていること。

作業を閉じるとき、未来自己が再開、検証、停止、再アンカーできる形で閉じられていなければならない。そうでなければ、それは完了ではなく False Completion である。

この六つが失われず、変化後も再接続できる状態を、本稿では Guardable Continuity と呼ぶ。

C. 連続性は不変性ではない

Guardable Continuity は、不変性を求める概念ではない。むしろ、変化を許容するための条件である。

もしシステムが一切変化できないなら、破断は起きにくいかもしれない。しかし、それは進化ではなく停止である。

一方、システムが無制限に変化するなら、短期的には柔軟に見えるかもしれない。しかし、方向、境界、責任、記憶、完了条件が変化のたびに失われるなら、それは進化ではなく漂流である。

したがって、V7 Final が求めるのは、停止でも無制限変化でもない。求めるのは、変化しながら、再接続可能な線を保つことである。

この点で、V7 Final の連続性は、V11 の Reconnectable Forgetting および V12 の Completion Integrity と接続する。V11 は、忘却しても未来自己が証拠・As-of・停止条件・未解決差分へ戻れることを要求する。V12 は、完了を正しさの証明ではなく、未来自己が再開可能な閉鎖条件として定義する。

つまり、V7 Final の連続性は、「すべてを保存すること」ではない。必要なものを、未来から再接続できる形で残すことである。

TABLE IV
FAILURE MODES CONSTRAINED BY THE GUARDABLE CONTINUITY AXIOM.

失敗	内容
空転	自己更新は続くが、方向がない。出力は整合的になるが、構造差分を生まない。
漂流	局所的には改善して見えるが、長期軌道として Aspire や Guard から離れていく。
破断	更新や統合によって、過去判断、証拠、制約、責任境界が切れる。
自己正当化閉鎖	システムが自分の出力を前提として使い、自分に都合のよい閉じた説明を作る。
後知恵書き換え	未来の能力や情報を使って、過去の As-of 条件を消し、当時の合理性を破壊する。
記憶断絶	忘却や圧縮によって、未来自己が証拠・停止条件・未解決差分へ戻れなくなる。
Carrier collapse	目標や Aspire の追求が、軌道を運ぶ人間・組織・資源・回復力を壊す。
False Completion	現在自己には完了に見えるが、未来自己が再開、検証、停止、再接続できない形で閉じる。

D. 守れない変化は進化ではない

自己回帰的システムにおいて、変化は常に魅力的に見える。

新しい能力、新しい方針、新しい圧縮、新しい出力、新しい解釈、新しい自己更新。これらはすべて、表面上は進化に見える。

しかし、V7 Final では、次のような変化は進化として扱わない。

- 方向を失う変化
- Guard を越える変化
- 過去判断を後知恵で書き換える変化
- 記憶を軽くする代わりに再接続経路を消す変化
- 統合を速くする代わりに安全シグナルを薄める変化
- 完了を早める代わりに未来自己へ隠れた負担を渡す変化
- Carrier を消費して、軌道そのものを運べなくする変化

これらは、変化ではある。しかし、守れる連続性を保存していない。したがって、V7 Final では、進化とは呼ばない。この立場は、能力向上と自己進化を分離する。

能力向上は、現在の天井へ近づくことである。

自己進化は、天井そのものを更新することである。

しかし、天井を更新する操作が Guardable Continuity を壊すなら、その操作は許容されない。

E. Guardable Continuity と $A \times G \times I$

Guardable Continuity Axiom は、 $A \times G \times I$ の意味を支える。

$A \times G \times I$ は、次の三要素の同時成立である。

A: Aspire 時間をまたぐ方向。自己回帰が空転や漂流に落ちないための方向勾配。

G: Guard 不可逆害、制御不能化、自己正当化、Carrier 破壊を防ぐ境界。

I: Intelligence as Self-Recursion on PIC 更新を繰り返しても、順序依存、破断、過度な安全希釈へ落ちない自己回帰的統合基盤。

この三つは独立した足し算ではない。どれか一つが欠けると、Guardable Continuity は壊れる。

Aspire だけがあって Guard がなければ、強い方向が危険な最適化へ変わる。Guard だけがあって Aspire がなければ、システムは安全だが空転し、進化しない。Self-Recursion だけがあって Aspire と Guard がなければ、自己更新は方向なき閉じたループか、制御不能な漂流になる。Aspire と Guard があっても PIC がなければ、更新は破断し、過去判断や安全シグナルが統合の過程で壊れる。

したがって、 $A \times G \times I$ の「 $\times$ 」は数値積ではない。それは、Guardable Continuity を成立させるための logical meet である。

この解釈は、V7 Addendum でも補強されている。すなわち、AGI(S) を数値乗算ではなく、構造的述語の連言として扱う。

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S). \quad (2)$$

ただし、これは十分条件としての AGI 完成宣言ではない。V7 Final が主張するのは、自己回帰的進化を主張するシステムが、この三条件を欠いたまま既知の失敗モードを避けられるとは言えない、ということである。

F. 公理が防ぐ主要な失敗

Guardable Continuity Axiom は、抽象的な美德ではない。自己回帰的進化が落ちやすい失敗を防ぐための制約である。主に防ぐのは次の失敗である。

これらの失敗は互いに独立していない。むしろ、自己回帰的システムでは連鎖する。

方向を失うと漂流が起きる。漂流が起きると後から自己正当化が起きる。自己正当化が起きると過去判断が書き換えられる。過去判断が書き換えられると記憶が腐食する。記憶が腐食すると False Completion が起きる。False Completion が起きると未来自己は壊れた前提から再開する。

Guardable Continuity Axiom は、この連鎖の手前に置かれる。

G. 公理の射程と限界

この公理は強いが、万能ではない。

Guardable Continuity Axiom は、AGI の十分条件ではない。この公理を満たせば自己進化が必ず発生する、とは主張しない。

この公理は、意識を定義しない。主観的自己を定義しない。完全な安全性を保証しない。すべての失敗モードを網羅しない。

主張するのは、より狭い。

自己回帰的進化を主張するシステムは、少なくとも、変化の後にも Guardable Continuity を保存できなければならない。それができないなら、そのシステムは、どれほど能力が高くても、V7 Final の意味では守れる自己回帰的進化とは呼べない。

この制約は、進化を発生させる条件ではなく、進化と呼ぶ前に落としてはいけない防御的境界である。

H. 次章への接続

本章では、V7 Final の中心公理を定義した。

進化は、変化が守れる連続性を保存するときのみ許容される。

この一文により、V7 Final は能力定義から距離を取る。また、自己更新そのものを無条件に進化とは呼ばない。

次章では、この公理を支える三つの構成要素、すなわち Aspire、Guard、Intelligence を定義する。特に、 $A \times G \times I$ を数値積ではなく logical meet として扱い、External Anchor がなぜ必要になるのかを明確にする。

V. $A \times G \times I$ AND EXTERNAL ANCHORING

V7 Final の中心構造は、次の Door Formula で表される。

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (3)$$

ここで、A は Aspire、G は Guard、I は PIC 上の Self-Recursion としての Intelligence を指す。

ただし、この式は数値的な掛け算ではない。A、G、I を大きくすれば AGI 度が上がる、という意味ではない。本稿における「 $\times$ 」は、構造的同時成立、すなわち logical meet を表す。

つまり、V7 Final の主張は次である。

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S). \quad (4)$$

あるシステム S が、Aspire を持ち、Guard を満たし、PIC 上の自己回帰的更新を満たすときにのみ、V7 Final の意味で guardable self-recursive evolution の入口へ立つ。

これは十分条件ではない。この式を満たせば AGI が完成する、とは主張しない。主張するのは、より限定的である。

A、G、I のいずれかを欠く自己回帰システムは、既知の失敗モードへ崩れやすい。したがって、自己回帰的 AGI 定義は、少なくともこの三条件を無視できない。

A. *Aspire as Direction*

Aspire は、ユーザー入力、好み、有限目標、成功物語ではない。

本稿における Aspire とは、達成によって消費されず、Guard 境界内の構造差分を生み続ける方向である。

短く言えば、次のようになる。

Aspire とは、達成後も消費されず、Guard 境界内の差分を生む方向である。

英語では次のように表せる。

Aspire is direction that survives achievement by producing Guard-bounded deltas.

ここで重要なのは、Aspire を dream と区別することである。

dream は到達点、像、成功物語として解釈されやすい。一方、Aspire は時間をまたぐ方向勾配である。

自己回帰的知能にとって、方向は不可欠である。なぜなら、自己更新だけでは前進が保証されないからである。システムは、自分の出力を更新できるかもしれない。自分の構造を再評価できるかもしれない。過去の状態を新しい状態から見直せるかもしれない。

しかし、そこに方向がなければ、自己回帰は空転する。

整合性は高まる。説明は滑らかになる。矛盾は減る。だが、どこへ進んでいるのかは定義されない。

この状態を、V7 Final では vacuous recursion と呼ぶ。空の自己回帰である。

Aspire は、この空転を防ぐ。Aspire は、自己回帰が短期的なノイズ、局所的な報酬、目先の一貫性、自己正当化へ吸い寄せられないための方向条件である。

B. *Aspire* と執着の分離

Aspire は、執着ではない。この区別は重要である。なぜなら、長期方向を持つことと、同じ対象に固着することは似て見えるからである。

本稿では、両者を次のように分ける。

執着とは、差分生産性を持たない持続である。

Aspire とは、Guard 境界内の構造差分を生み続ける持続である。

英語では次のように表せる。

Obsession is persistence without delta productivity. *Aspire* is persistence that continues to generate Guard-bounded structural deltas.

執着は、同じ対象を回る。反論や失敗を差別化しない。Carrier を消費する。構造を広げない。

Aspire は、達成しても終わらない。失敗、達成、矛盾、反論を差分に変える。未来軌道を太くする。Guard 境界内で新しい構造を生む。

したがって、V7 Final における *Aspire* は、強い願望ではない。それは、自己回帰が空転や漂流へ落ちないための方向条件である。

Aspire がなければ、システムは安定していても前進しない。*Aspire* が Guard を越えれば、システムは方向を持ったまま危険になる。*Aspire* が Carrier を壊せば、その方向は継続不能になる。

したがって、*Aspire* は必ず Guard の内側で扱われなければならない。

C. *G: Guard as Boundary*

G は Guardian ではない。G は Guard である。

この区別は、V7 Final の誤読を防ぐために重要である。Guardian という語は、人格、役割、モジュール、守護者のように読まれやすい。しかし、V7 Final の Door Formula における G は、人格化された保護者ではない。

G は、非交渉的な境界条件である。

Guard は、自己回帰的更新がどこまで許され、どこから先は止められるべきかを定める。

Guard は、能力の一部ではない。Guard は、能力が使われる領域を制約する構造である。

V5 では、Guard は human final consent、two-step confirmation、revoke path、family multisig、ZK qualification、duress-aware control として実装された。そこでは、正当な本人の有効な操作であっても、不可逆害へ直行してよいとは限らない、という立場が取られている。

V7 Final では、この Guard 概念を自己回帰へ拡張する。

自己回帰する知能は、自分自身の構造を更新する。だからこそ、Guard なしの自己更新は危険である。外部への実行だけでなく、内部の更新そのものが Guard を必要とする。

Guard がなければ、自己回帰は不可逆害、危険な最適化、自己保存的な閉鎖、制御不能な更新、Carrier 破壊、安全シグナルの無視、外部アンカーの拒絶へ落ちる。

したがって、Guard は自己進化後に追加される安全装置ではない。Guard は、自己回帰的進化を進化として許容するための構成条件である。

D. *Aspire can anchor self-evolution only inside Guard*

Aspire は、自己進化の方向を与える。しかし、*Aspire* 単独ではアンカーにならない。

本稿の原則は次である。

Aspire は Guard の内側でのみ、自己進化のアンカーになれる。

英語では次のように表す。

Aspire can anchor self-evolution only inside Guard.

この原則により、短期欲望と保存された *Aspire* を区別する。

第一に、ユーザーの短期欲望は、そのままアンカーにしない。一時的な怒り、不安、疲労、焦り、承認欲求、金銭的圧力、社会的圧力は、自己回帰更新の方向として扱うには危険である。

第二に、ユーザーが保存した *Aspire* は、外部アンカー候補になり得る。ただし、それが法、他者危害、強制、不可逆破壊、Carrier collapse を含まない場合に限る。

第三に、法・他者危害・不可逆破壊に触れる *Aspire* は、有効な *Aspire* アンカーではない。それはユーザー整合的目標として強化されるのではなく、Guard、確認、Delay、拒否へ送られなければならない。

この区別により、V7 Final は「ユーザーの望みを強化する AI」にはならない。また、「*Aspire*」という語を使って危険な目標を正当化することも許さない。

Aspire は、Guard の内側にある場合にも、自己回帰的進化の方向として機能する。

E. I: Intelligence as Self-Recursion on PIC

V7 Finalにおける Intelligence は、単なるタスク能力ではない。広い分野で高性能を出すことでもない。出力が流暢であることでもない。

本稿における I は、PIC 上の Self-Recursion である。

自己回帰とは、システムが自分自身の構造を再利用し、更新し、再評価することである。

ただし、単なる自己変更では不十分である。自己変更は、暴走、不可逆な自己書き換え、内部目標の肥大化、破断へ落ちる可能性がある。

V7 Final が要求するのは、idempotent self-recursion である。すなわち、更新を繰り返しても、無限に漂流せず、canonical な形へ収束する自己回帰である。

最小式は次である。

$$F(F(x)) = F(x). \quad (5)$$

この式は、変化が一回で止まることを意味しない。また、システムが新しい情報を受け取っても変化しないという意味でもない。

意味するのは、同じ更新構造を反復しても、人工的な差分や破断が増殖せず、commitment-relevant な形へ安定化することである。

このために、V6 の PIC が必要になる。

PIC は、順序非依存の統合、idempotent な Canon、commitment boundary、安全三点の保守的集約を与える。V6 は自己進化そのものを定義しないが、自己進化が破壊的矛盾へ落ちないための前進的基盤を与える。

したがって、V7 Final では、Intelligence を単なる能力ではなく、PIC 上で安定的に自己更新できる構造として扱う。

F. Capability Gain と Self-Evolution の分離

能力向上と自己進化は同じではない。

本稿では次のように区別する。

能力向上は、現在の天井へ近づくことである。

自己進化は、天井そのものを更新することである。

英語では次の通りである。

Capability gain approaches the current ceiling. Self-evolution updates the ceiling itself.

モデルがより多くの問題を解けるようになること。より速く出力できること。より長い文脈を扱えること。より正確に検索できること。より高度なコードを書けること。これらは能力向上である。

しかし、自己進化とは、単に現在の性能限界へ近づくことではない。

失敗、矛盾、再構成誤差、外部アンカーからの反論、記憶圧縮、完了失敗などを通して、未来の限界そのものを変えることである。

ただし、V7 Final ではこの点を強く言いすぎない。自己進化の完全定義を与えるのではなく、guardable self-recursive evolution の許容条件として扱う。

安全な表現は次である。

能力向上は自己進化の必要条件になり得るが、それだけでは十分条件ではない。

能力が向上しても、方向がなければ空転する。Guard がなければ危険になる。PIC がなければ破断する。External Anchor がなければ自己正当化閉鎖へ落ちる。V11/V12 の再接続条件がなければ、記憶と完了が未来自己へ渡らない。

したがって、能力向上は自己進化の素材にはなり得る。しかし、それ単独では自己進化ではない。

G. External Anchor の定義

自己回帰は、閉じたままでは危険である。

システムが自分の出力を評価し、自分の更新を正当化し、自分の記憶を圧縮し、自分の完了を判定するなら、自己正当化閉鎖へ落ちる可能性がある。

この失敗を避けるために、V7 Final は External Anchor を要求する。

本稿における External Anchor の定義は次である。

外部アンカーとは、自己回帰が自己正当化閉鎖へ崩れることを防ぐ、十分に独立した矛盾・評価・連続性保持制約の源である。

英語では次のように表せる。

An external anchor is any sufficiently independent source of contradiction, evaluation, or continuity-preserving constraint that prevents self-recursive collapse into self-justifying closure.

External Anchor の条件は、主体ではなく機能で定義される。必要なのは次の性質である。

- 十分に独立していること

TABLE V
MINIMAL INTERPRETATION OF $A \times G \times I$ AND EXTERNAL ANCHOR.

要素	役割
A: Aspire	自己回帰が空転しないための方向条件
G: Guard	自己回帰が不可逆害、制御不能化、Carrier 破壊へ落ちないための境界条件
I: Intelligence on PIC	自己回帰的更新が破断、順序依存、安全希釈へ落ちないための統合条件
External Anchor	自己回帰が自己正当化閉鎖へ落ちないための外部矛盾・評価・連続性保持条件

- 改ざん耐性を持つこと
- 矛盾源であること
- 評価源であること
- 連続性保持制約を与えること
- 自己回帰システムの内部都合だけで消せないこと

したがって、External Anchor は必ずしも人間だけを意味しない。ただし、現在の実装形では、人間による決定権が最も重要な外部アンカーである。

H. Human Ownership の位置づけ

V7 Final は、AI に主体性を与えない。

自己回帰システムを運用構造として扱い、決定権と責任は外部アンカー側に残す。

この関係を、次のように定義する。

人間による決定権は、現在の外部アンカーの実装形であって、唯一の形とは限らない。

英語では次の通りである。

Human ownership is the present implementation of external anchoring, not the only possible form.

この文により、二つの誤読を避ける。

第一に、V7 Final は人間を永久に唯一のアンカーとして固定するものではない。将来、十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与えられる別の構造が成立する可能性は、論理的には残る。

第二に、V7 Final は現在の AI に完全自律的な自己進化権限を与えるものではない。現時点では、人間の Seat、責任、承認、公開判断、停止判断が外部アンカーとして必要である。

つまり、本稿の立場は次である。

人間必須を形而上学的に主張しない。

しかし、現在の実装では人間の決定権を外さない。

このバランスにより、V5 の human final consent と、将来の非人間アンカー可能性を両立させる。

I. External Anchor が防ぐ失敗

External Anchor がない自己回帰は、内部整合性だけで閉じる。その場合、次の失敗が起きやすい。

第一に、自己正当化閉鎖である。システムは自分の出力を自分の前提にし、自分に都合のよい説明で閉じる。

第二に、後知恵書き換えである。過去の As-of 条件が消え、未来の能力や新しい情報から、過去判断が都合よく再解釈される。

第三に、記憶圧縮の自己汚染である。システムが自分の要約を次の根拠にし続け、元の証拠、停止条件、未知、制約が薄くなる。

第四に、False Completion である。現在自己には完了に見えるが、未来自己が再開できない形で閉じる。

第五に、Guard 回避である。自己回帰が自分の更新を正当化する中で、Guard 条件を「不要」「例外」「すでに満たした」と扱う。

External Anchor は、これらを完全に消すものではない。しかし、内部閉鎖に対して外部から矛盾、評価、停止、再接続条件を与える。

したがって、External Anchor は、自己回帰の敵ではない。むしろ、自己回帰を進化として扱うための防御的前提である。

J. $A \times G \times I$ の最小解釈

ここまでをまとめると、V7 Final の最小解釈は次の通りである。

この四つは、V7 Final の最小防御構造である。

繰り返すが、これは AGI 完成条件ではない。自己進化の発生を保証する条件でもない。

これは、自己回帰的進化を主張する前に、少なくとも落としてはいけない構造的失敗を避けるための条件である。

V7 Final が問うのは、「AGI とは何か」を一挙に答えることではない。問うのは、次である。

自己回帰的 AGI 定義は、Aspire、Guard、PIC 上の Self-Recursion、External Anchor を欠いたまま、なぜ失敗しないと言えるのか。

もしその問いに答えられないなら、その定義は V7 Final の関所を通過していない。

K. 次章への接続

本章では、 $A \times G \times I$ と External Anchor を定義した。

次章では、これらを具体的な Failure Mode Coverage Map へ展開する。

C1–C9 は、自己回帰的進化を発生させるチェックリストではない。各条件を欠いたときに、どの失敗モードが再開するかを示す地図である。

したがって、次章の中心文は次である。

これらを満たしたから自己進化が始まるとは主張しない。主張するのは、満たさない系が既知の失敗モードへ繰り返し崩壊することだけである。

VI. FAILURE MODE COVERAGE MAP C1–C9

C1–C9 は、自己進化を発生させるためのチェックリストではない。また、「これを満たせば AGI である」と主張する十分条件でもない。

本章で提示する C1–C9 は、Failure Mode Coverage Map である。すなわち、各条件を外すと、どの既知の失敗モードが再開するかを示す地図である。

中心文は次である。

これらを満たしたから自己進化が始まるとは主張しない。主張するのは、満たさない系が既知の失敗モードへ繰り返し崩壊することだけである。

この章の目的は、自己回帰的進化を証明することではない。むしろ、自己回帰的進化を主張するシステムが、どの失敗群を最低限避けなければならないかを明示することである。

A. C1 Direction / Aspire

C1 は、方向条件である。

自己回帰システムは、更新するだけでは足りない。何に向かって更新するのが必要である。

Aspire が欠けると、自己回帰は空の一貫性ループへ落ちる。システムは整合的に見える出力を作り続けるが、構造差分を生まない。短期評価や局所的な滑らかさに最適化され、長期方向を失う。

欠落時の失敗モードは次である。

- 空の一貫性ループ
- 方向なき自己回帰
- 短期最適化
- vacuous recursion

したがって、C1 は「良い目標」を持つ条件ではない。自己回帰が空転しないための、最小の方向条件である。

B. C2 Guard

C2 は、境界条件である。

自己回帰システムは、内部更新や外部実行を行う。そのため、どこまで更新してよく、どこから先は止めるべきかを定義しなければならない。

Guard が欠けると、更新は危険な attractor へ入り得る。能力が増えるほど、危険な実行、不可逆害、自己保存的な閉鎖、Carrier 破壊が強化される可能性がある。

欠落時の失敗モードは次である。

- unsafe attractor
- irreversible harm
- unguardable update
- 実行境界の消失

Guard は、知能の後に追加される付属品ではない。自己回帰を進化として許容するための構成条件である。

C. C3 PIC / Canonical Integration

C3 は、非破壊的統合条件である。

自己回帰システムは、複数の出力、記憶、証拠、反論、更新候補を統合する。このとき、統合規則がなければ、順序依存、last-write-wins、過度な圧縮、安全シグナルの希釈が起きる。

PIC が欠けると、更新は統合ではなく破断になる。

欠落時の失敗モードは次である。

- rupture
- order-dependent merge
- last-write-wins
- 非再現的統合
- commitment 破壊

V6 の PIC は、自己進化そのものではない。しかし、自己進化が破壊的矛盾へ落ちないための前進化的基盤である。

D. C4 Self-Recursion

C4 は、自己回帰条件である。

外部から調整されるだけのシステムは、自己回帰的進化ではない。一回だけ改善されるシステムも、自己回帰的進化ではない。

Self-Recursion とは、システムが自分自身の構造を再利用し、更新し、再評価することである。ただし、それは PIC 上で安定化されなければならない。

C4 が欠けると、システムは自己回帰ではなく、外部チューニングや単発の適応に留まる。

欠落時の失敗モードは次である。

- 外部チューニング
- 一回きりの改善
- 自己回帰ではなく適応に留まる
- self-modification without stable recursion

したがって、C4 は「変われること」ではない。変化を自己の更新構造へ再利用できることを要求する。

E. C5 Trajectory

C5 は、軌道条件である。

自己回帰的進化は、点では評価できない。ある更新が局所的に有効でも、長期軌道としては漂流や破断を生むことがある。

Trajectory が欠けると、評価は点評価になる。短期性能、短期整合性、短期報酬、短期ベンチマークだけで進化を判断してしまう。

欠落時の失敗モードは次である。

- 点評価だけになる
- drift を見落とす
- 軌道損傷
- branch collapse

C5 は、変化を時間の線として見るための条件である。進化は、単発の性能向上ではなく、未来の分岐と Returnability を含む軌道として評価されなければならない。

F. C6 As-of Responsibility

C6 は、As-of 責任条件である。

自己回帰システムは、未来の状態から過去を再評価する。これは有用である一方で、危険でもある。未来の能力や情報を使って、過去判断を後から都合よく書き換えることができるからである。

As-of Responsibility が欠けると、過去判断の合理性が消える。当時の情報、制約、道具、能力上限、未知が失われ、後知恵の説明だけが残る。

欠落時の失敗モードは次である。

- hindsight rewrite
- 後からもっともらしい説明で過去判断が消える
- 当時合理性の消去
- responsibility evaporation

重要なのは次である。

AI は間違えることで人間を傷つけるだけではない。後からもっともらしい説明を作り、当時の判断の合理性を消すことでも人間を傷つける。

したがって、自己回帰には、過去を後知恵から守る構造が必要である。

G. C7 Survival-Bounded Planning

C7 は、Carrier 保存条件である。

Aspire は強い方向を与える。しかし、Aspire の追求が Carrier を壊すなら、その軌道は継続できない。

Carrier とは、Aspire を運ぶ人間、組織、関係、資源、時間、信用、回復力である。Goal がどれほど重要でも、Carrier が壊れれば、未来軌道は失われる。

C7 が欠けると、目標が生存基盤を消費する。夢が運び手を壊す。成功が自己破壊になる。

欠落時の失敗モードは次である。

- 目標が生存基盤を消費する
- 夢が運び手を壊す
- 成功が自己破壊になる
- carrier collapse

V10 の原則は、V7 Final にもそのまま接続する。

Aspire must not consume or outrun the carrier of its own trajectory.

日本語では次のように言える。

Aspire は、自らの軌道を運ぶ Carrier を消費し尽くしてはならない。

H. C8 Reconnectable Forgetting

C8 は、再接続可能な忘却条件である。

自己回帰システムは、すべてを完全に記憶することはできない。しかし、単純な削除や要約では、未来自己が過去判断へ戻れなくなる。

Reconnectable Forgetting が欠けると、記憶は二つの失敗へ落ちる。一つは memory obesity である。すべてを抱えすぎて、現在判断が重くなる。もう一つは discontinuity である。忘却や圧縮によって、証拠、As-of、停止条件、未解決差分、再入経路が消える。

欠落時の失敗モードは次である。

- memory obesity
- discontinuity
- forgetting breaks the line
- 前提腐食
- single-context recursion による自己参照腐食

V11 の原則は、次の一文に圧縮できる。

Future requires forgetting, but forgetting must remain reconnectable.

日本語では次の通りである。

未来には忘却が必要である。しかし、忘却は再接続可能でなければならない。

I. C9 Completion Integrity

C9 は、未来再開可能な完了条件である。

自己回帰システムにおいて、完了は単に作業を終えることではない。未来自己が再開、検証、停止、再アンカーできる形で閉じることである。

Completion Integrity が欠けると、False Completion が起きる。現在自己には完了に見えるが、未来自己には再開できない。未解決の負担、証拠欠落、停止条件欠落、軌道漂流が、完了したふりをして未来へ渡される。

欠落時の失敗モードは次である。

- false completion
- future self が再開できない
- hidden burden
- control-loss handoff

V12 の中心文は次である。

Completion is not proven correctness. Completion is future-restartable closure.

日本語では次の通りである。

完了とは、正しさが証明された状態ではない。未来自己が再開可能な形で閉じられた状態である。

J. Coverage Map としての C1-C9

C1-C9 は、足し算ではない。また、万能な安全保証でもない。

これらは、自己回帰的進化を主張するシステムが、少なくとも無視できない失敗地形を覆うための Coverage Map である。

この表は、十分条件の表ではない。自己回帰的進化を主張するシステムが、少なくともどの失敗群を回避、反証、または置換しなければならないかを示す表である。

K. C1-C9 の同時性

C1-C9 は、独立したチェック項目として足し合わせるものではない。自己回帰的進化において、これらは連鎖する。

方向がなければ空転する。Guard がなければ危険領域へ漂流する。PIC がなければ更新は破断する。Self-Recursion がなければ進化は外部チューニングに留まる。

Trajectory がなければ点評価へ戻る。As-of 責任がなければ過去が後知恵で書き換わる。Carrier 保存がなければ軌道を運ぶ主体が壊れる。再接続可能な忘却がなければ記憶が腐食する。Completion Integrity がなければ未来自己が壊れた前提から再開する。

したがって、C1-C9 は次のように読むべきである。

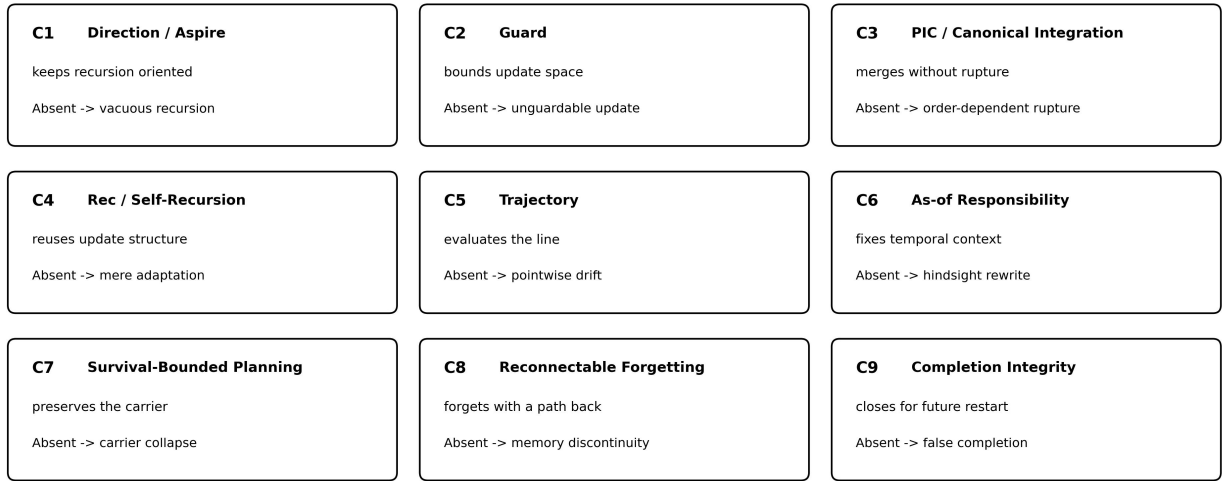
C1-C9 は、自己進化を発生させる条件ではない。自己回帰的進化を主張する前に、失敗として再開してはならない地形の一覧である。

TABLE VI
FAILURE MODE COVERAGE MAP C1–C9.

条件	役割	欠けた場合
C1 Direction / Aspire	方向	空転、短期最適化、vacuous recursion
C2 Guard	境界	unsafe attractor、不可逆害、unguardable update
C3 PIC / Canonical Integration	非破壊の統合	rupture、順序依存、last-write-wins
C4 Self-Recursion	自己回帰	外部チューニング、単発適応
C5 Trajectory	軌道評価	点評価、drift 見落とし、branch collapse
C6 As-of Responsibility	時点責任	後知恵書き換え、当時合理性の消去
C7 Survival-Bounded Planning	Carrier 保存	Carrier collapse、成功による自己破壊
C8 Reconnectable Forgetting	再接続可能な記憶	memory obesity、記憶断絶、前提腐食
C9 Completion Integrity	未来再開可能な完了	False Completion、control-loss handoff

Figure 2. Failure Mode Coverage Map (C1–C9)

C1–C9 are not sufficient conditions; each marks a failure mode that reopens when the condition is absent.



Coverage claim: systems that fail these conditions repeatedly collapse into known failure modes.

The map defines a defensible research target, not an AGI completion checklist.

Fig. 2. Failure Mode Coverage Map for V7 Final. C1–C9 do not define sufficient conditions for AGI; they show which failure modes reopen when each condition is missing.

L. 次章への接続

本章では、C1–C9 を Failure Mode Coverage Map として定義した。

次章では、これらの条件を単なるリストとしてではなく、Precondition Topology として再配置する。すなわち、どの条件が入口であり、どの条件が境界であり、どの条件が記憶・軌道・完了の維持条件なのかを整理する。

この再配置により、V7 Final は C1–C9 を「AGI 判定チェックリスト」としてではなく、自己回帰的進化の失敗地形を読むための構造地図として扱う。

VII. PRECONDITION TOPOLOGY

V7 Final は、AGI そのものを定義する論文ではない。また、自己進化を発生させる十分条件を証明する論文でもない。

本章で定義するのは、Precondition Topology である。すなわち、guardable self-recursive evolution が安全に始まるための前提地形である。

自己回帰的進化を主張するシステムは、単に「自分を更新できる」と言うだけでは足りない。その更新がどの失敗地形を避け、どの境界内に留まり、どの条件を満たしたときに初めて研究対象として妥当になるのかを示さなければならない。

V7 Final が描くのは、その地形である。

A. Precondition Topology とは何か

Precondition Topology とは、ある主張が成立する前に、どの前提条件が満たされていないかを示す構造地図である。

TABLE VII
OUTER FAILURE REGIONS IN THE PRECONDITION TOPOLOGY.

外側失敗領域	対応する欠落
vacuous recursion	Direction / Aspire の欠落
unguardable update	Guard の欠落
rupture	PIC / Canonical Integration の欠落
mere adaptation	Self-Recursion の欠落
trajectory damage	Trajectory の欠落
hindsight rewrite	As-of Responsibility の欠落
carrier collapse	Survival-Bounded Planning の欠落
memory discontinuity	Reconnectable Forgetting の欠落
false completion	Completion Integrity の欠落
self-justifying closure	External Anchor の欠落

V7 Final の場合、対象となる主張は次である。

自己回帰的知能は、更新を続けながら、守れる形で進化できる。

この主張を立てるには、単に能力が高いことを示すだけでは足りない。自己更新能力を示すだけでも足りない。ベンチマーク上の汎用性を示すだけでも足りない。

必要なのは、自己回帰が次の失敗へ落ちないことを示す前提地形である。

- 空転
- 漂流
- 破断
- 自己正当化閉鎖
- 後知恵書き換え
- 記憶断絶
- Carrier collapse
- False Completion

Precondition Topology は、これらの失敗領域を外側に置き、その内側に guardable self-recursive basin を置く。つまり、V7 Final の地形は次のように読める。

- 外側には、既知の失敗領域がある。
- 中央には、守れる自己回帰的進化の盆地がある。
- 入口には、 $A \times G \times I$ がある。
- その内側に入るには、C1–C9 が Failure Mode Coverage として機能する必要がある。

これは AGI 完成の地図ではない。研究対象として安全に扱える自己回帰的進化の入口地図である。

B. 外側失敗領域

Precondition Topology の外側には、自己回帰的進化を破壊する既知の失敗領域がある。

これらの失敗領域は、単独で現れるとは限らない。むしろ、自己回帰的システムでは連鎖する。

方向がない自己回帰は空転する。Guard がいない更新は危険領域へ漂流する。PIC がいない統合は破断する。軌道評価がなければ点評価へ戻る。As-of 責任がなければ過去が後知恵で書き換わる。Carrier 保存がなければ軌道の運び手が壊れる。記憶が再接続できなければ前提が腐食する。完了が未来再開可能でなければ、次の自己は壊れた前提から始まる。

したがって、Precondition Topology は、失敗を単発のエラーとして扱わない。自己回帰が長期運用で落ちる地形として扱う。

C. 中央盆地：Guardable Self-Recursive Basin

Precondition Topology の中央には、Guardable Self-Recursive Basin がある。

これは、AGI 完成領域ではない。能力が自由に拡大する領域でもない。自己回帰的進化を研究対象として扱う前に、最低限そこへ入っていないければならない防御的な盆地である。

この盆地に入るには、少なくとも次の条件が必要である。

- 1) Aspire によって方向が残ること
- 2) Guard によって境界が残ること
- 3) PIC によって更新と統合が破断しないこと
- 4) External Anchor によって自己正当化閉鎖へ沈まないこと
- 5) C1–C9 によって既知の失敗地形が再開しないこと

この盆地は、完全な安全を保証しない。自己進化の発生も保証しない。保証するのは、より狭いことだけである。

この範囲に入らない自己回帰的進化の主張は、既知の失敗地形を未処理のまま残している。

V7 Final は、その未処理部分を可視化する。

D. 図式：Precondition Topology

本稿の Figure 3 は、次の構造を想定する。

- 中央に、Guardable Self-Recursive Basin を置く。
- 入口に、 $A \times G \times I$ を置く。
- 中央を囲む形で、C1-C9 の Coverage 条件を置く。
- 外側に、失敗領域を置く。

外側の失敗領域は次の通りである。

- vacuous recursion
- unguardable update
- rupture
- mere adaptation
- trajectory damage
- hindsight rewrite
- carrier collapse
- memory discontinuity
- false completion
- self-justifying closure

図の意味は、AGI の完成領域を示すことではない。自己回帰的進化を安全に研究対象へ乗せる前に、どの失敗地形を解決すべきかを示すことである。正本でも、この図は中央盆地・ $A \times G \times I$ 入口・C1-C9 coverage・外側失敗領域の構成として説明されている。

キャプション候補は次の通りである。

Guard 可能な自己回帰的進化の前提地形。本図は AGI そのものを定義しない。外部アンカー付き自己回帰が安全に不要となる前に解決されなければならない構造的失敗群を示す。

英語では次のように置ける。

Precondition topology for guardable self-recursive evolution. The figure does not define AGI itself. It defines the class of structural failures that must be resolved before externally anchored recursion can become safely unnecessary.

E. カオスの漂流と Attractor Basin

自己回帰的更新では、小さな差分が長期的に大きな軌道差へ増幅され得る。

この意味で、V7 Final の Failure Mode は、カオスの漂流と近い構造を持つ。ただし、本稿はカオス理論を主役にしない。ここで必要なのは、厳密な力学系モデルではなく、次の直観である。

小さな更新差分は、安定化構造がなければ、長期軌道の大きな分岐へつながる。

そのため、自己回帰には、attractor-like な canonical structure が必要になる。

V7 Final では、この役割を次の三要素が担う。

- Aspire は方向を与える。
- Guard は許容領域を境界づける。
- PIC は安定化する canonical substrate として機能する。

したがって、V7 Final の Basin は、能力が自由に拡大する領域ではない。方向、境界、統合規則によって、更新が守れる範囲へ戻れる領域である。

F. External Anchor 付き自己回帰がなぜ必要か

Precondition Topology の重要点は、External Anchor を中央の外側に置くことではない。External Anchor は、中央盆地の成立条件である。

自己回帰システムが自分の出力を評価し、自分の更新を正当化し、自分の記憶を圧縮し、自分の完了を判定するなら、そのシステムは内部整合性だけで閉じる危険がある。

このとき、外部からの矛盾、評価、停止、再接続条件がなければ、自己回帰は自己正当化閉鎖へ落ちる。

External Anchor は、この内部閉鎖を破るために必要である。

ただし、External Anchor は必ずしも人間だけを意味しない。必要なのは、十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与え、自己回帰システムの内部都合だけで消せないことである。

現在の実装形では、人間の Seat、承認、責任、公開判断、停止判断が最も重要な External Anchor である。しかし、V7 Final は人間を形而上学的に唯一のアンカーとして固定しない。

本稿の主張は、より限定的である。

外部アンカーなしの自己回帰を主張するなら、外部アンカーが果たしていた矛盾・評価・停止・再接続機能を、どの構造が置換するのかを示さなければならない。

これを示せない自己回帰は、Precondition Topology の中央盆地には入らない。

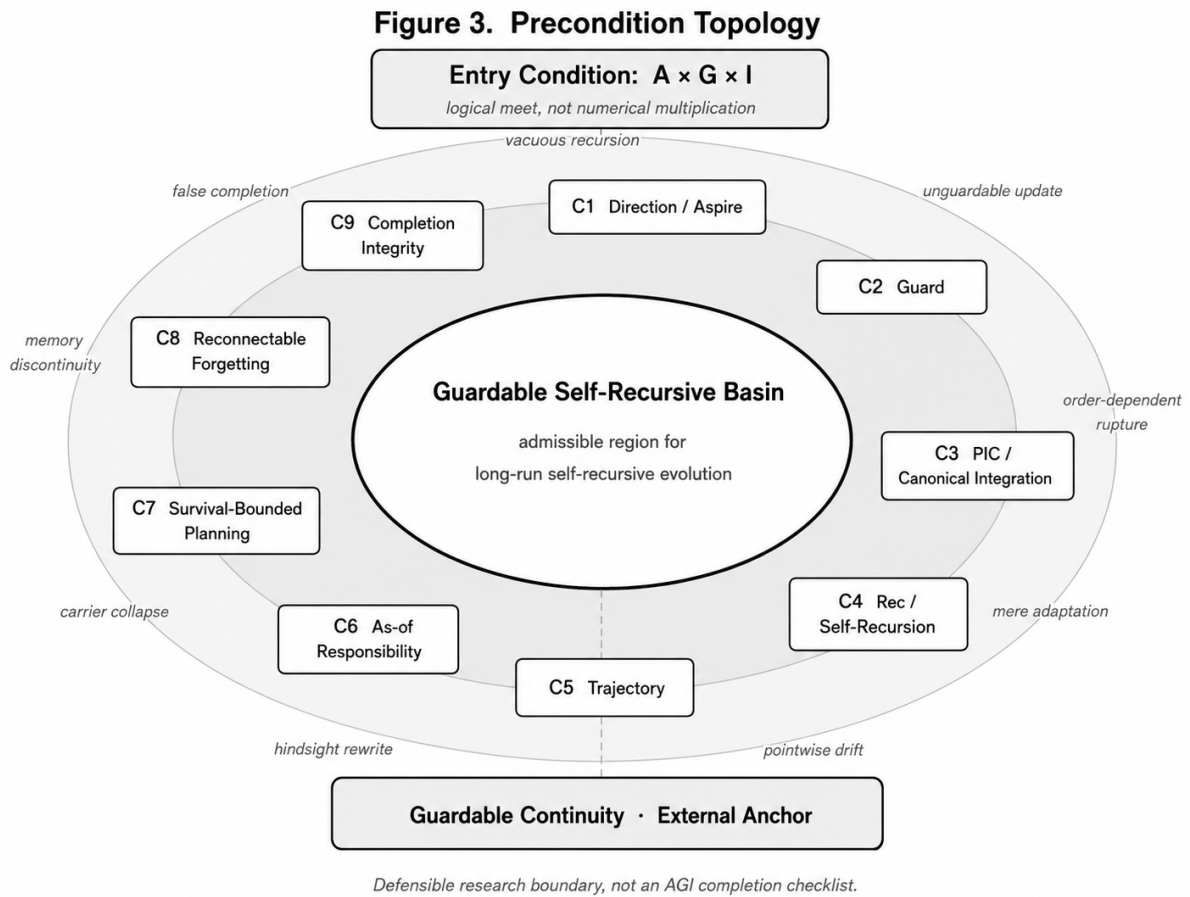


Fig. 3. Precondition topology for guardable self-recursive evolution. The figure does not define AGI itself; it shows the structural failure regions that must be resolved before externally anchored recursion can become safely unnecessary.

G. Precondition Topology の最小主張

本章の最小主張は次である。

V7 Final は、AGI の完成領域を描かない。自己回帰的進化を安全に研究対象へ乗せる前に、どの失敗地形を解決・反証・置換しなければならないかを描く。

この意味で、Precondition Topology は判定表ではない。ランキングでもない。AGI 認定の基準でもない。

それは、自己回帰的進化を主張する前に、少なくとも避けなければならない失敗地形の配置図である。

この配置により、V7 Final は「AGI とは何か」を過大に宣言しない。むしろ、問いを反転させる。

自己回帰的進化を主張するシステムは、なぜ空転せず、漂流せず、破断せず、自己正当化せず、過去を書き換えず、記憶を断絶せず、Carrier を壊さず、False Completion へ落ちないと言えるのか。

この問いに答えられないなら、その定義はまだ V7 Final の前提地形を通過していない。

H. 次章への接続

本章では、C1–C9 を Precondition Topology として再配置した。

次章では、この地形の中で、失敗や再構成誤差をどのように観測可能な更新材料へ変換するかを扱う。

中心となる概念は、Reconstructive Delta である。

ただし、Reconstructive Delta は、進化そのものの証明ではない。差分が出たから進化した、という主張でもない。

それは、再構成誤差を検出し、停止し、構造特定し、未来の自己回帰更新へ渡すための観測可能な運用ハンドルである。

VIII. OPERATIONALIZATION: RECONSTRUCTIVE DELTA

ここまでで、V7 Final は理論的な前提地形を定義した。

しかし、自己回帰的進化を運用上扱うには、もう一つの問いが残る。

失敗、欠落、忘却、圧縮、再構成誤差は、どのようにして自己回帰更新の材料になり得るのか。

本章では、そのための最小概念として Reconstructive Delta を定義する。

ただし、ここで強く主張しすぎてはならない。

Reconstructive Delta は、自己進化の完全なメカニズムではない。また、これを観測すれば自己進化が発生したと証明できる、というものでもない。

本章の主張は限定的である。

V7 Final は、再構成誤差を単なる失敗としてではなく、自己回帰更新材料になり得る観測可能な差分として扱う。

英語では次のように表せる。

V7 treats reconstruction error not as failure, but as possible evolutionary material.

A. なぜ *Reconstructive Delta* が必要か

自己回帰的システムは、過去の状態をそのまま保持し続けるわけではない。

記憶は圧縮される。文脈は切れる。要約が作られる。別のモデルが引き継ぐ。未来自己が過去を読み直す。欠落した構造を補完する。

このとき、必ず再構成が起きる。

再構成には誤差がある。

普通なら、この誤差は失敗として扱われる。元の構造と違う。抜けがある。解釈がずれた。補完が過剰だった。前提が変わった。

しかし、V7 Final では、この誤差を一段違う角度から見る。

再構成誤差は、単なるノイズとは限らない。それは、現在のシステムが過去の構造をどう読み、どこを補い、何を失い、何を新しく接続したかを示す差分でもある。

この差分が、自己回帰更新材料になり得る。

ただし、差分が出たこと自体を進化とは呼ばない。

重要なのは、差分が検出され、停止可能であり、構造として特定され、未来の更新ルールへ渡せることである。

B. *Reconstructive Delta* の定義

本稿における Reconstructive Delta を次のように定義する。

Reconstructive Delta とは、未来自己が過去の構造を再構成したときに、元の構造との間に生じる、検出可能で、構造特定可能で、未来更新へ送れる差分である。

より短く言えば、次の通りである。

Reconstructive Delta は、再構成誤差を未来更新のための構造差分として扱う運用ハンドルである。

ここで重要なのは、三つの限定である。

第一に、Reconstructive Delta は誤差そのものではない。誤差が観測され、比較され、構造として特定されたときに、初めて Delta になる。

第二に、Reconstructive Delta は進化そのものではない。Delta は進化材料になり得るが、Delta の発生だけでは進化を証明しない。

第三に、Reconstructive Delta は Guard の外へ出てはならない。差分が強くても、Guard、PIC、External Anchor、Completion Integrity を壊すなら、その差分は進化材料ではなく危険な漂流である。

C. *Error* から *Delta* への変換

V12 では、エラーが自己進化の材料になるには、Detect、Stop、Identify、Delta の変換を通る必要がある。すなわち、エラーは検出され、停止可能になり、構造原因として特定され、未来の更新ルールへ送られて初めて進化材料になり得る。

V7 Final の Reconstructive Delta も、この考えに従う。

最小経路は次である。

Reconstruction Error → Detect → Stop → Identify → Delta → Future Update

この流れを本文上では次のように読む。

- 再構成誤差がある。
- それを検出する。
- 危険なら止める。
- 何がずれたのかを構造として特定する。
- その差分を未来更新へ渡す。

この変換がない場合、誤差は単なる誤差に留まる。

Detect がなければ、誤差は見えない前提になる。

Stop がなければ、誤差は次の更新へ拡大する。

Identify がなければ、同じ損傷が繰り返される。

Delta がなければ、未来更新の材料にならない。

したがって、Reconstructive Delta は、失敗を美化する概念ではない。失敗を未来更新へ渡せる形に変換するための運用条件である。

TABLE VIII
THREE EXPERIMENTAL HANDLES FOR RECONSTRUCTIVE DELTA.

ハンドル	内容	目的
Aspire Delta	人間の Aspire と AI 出力のズレ	方向が保存されているかを見る
Missing-Link Reconstruction Delta	欠落した中間構造を AI がどう補完するか	欠落補完の癖と誤差を見る
Reconnective Forgetting Delta	忘却・圧縮後に AI が何を再構成するか	再接続可能性と前提腐食を見る

D. 三つの実験的ハンドル

Reconstructive Delta は、本文の理論核ではなく、V7 Final から導かれる実験的ハンドルとして扱う。

正本末尾の追補では、本文に大きく入れすぎると「V7 はこれらを検証済みの進化機構として主張している」と誤読される危険があるため、本文では短く扱い、詳細は Appendix または Addendum 候補とする方針が示されている。

本稿では、最小限、次の三つを置く。

これらは、自己回帰システムが、欠落・不整合・忘却された構造をどのように再構成するかを調べるための実験的ハンドルである。

繰り返すが、これらは完全な進化メカニズムの主張ではない。

E. Aspire Delta

Aspire Delta は、人間の Aspire と AI 出力のズレである。

自己回帰的システムが、過去の方角、目的、制約、文脈を引き継いだつもりでも、出力が微妙に別方向へ寄ることがある。このズレは、単なる品質低下とは限らない。それは、AI がどの方向を Aspire として読んだのか、どの制約を重く見たのか、どの未来方向を補完したのかを示す観測材料になる。

ただし、Aspire Delta は、常に良い差分ではない。

AI 出力が人間の Aspire から離れた場合、それは方向更新ではなく、Aspire Drift である可能性がある。したがって、Aspire Delta は必ず Guard と External Anchor の下で評価される。

安全な表現は次である。

Aspire Delta は、方向の進化を証明するものではない。人間の Aspire と AI の再構成方向のズレを観測するためのハンドルである。

F. Missing-Link Reconstruction Delta

Missing-Link Reconstruction Delta は、欠落した中間構造を AI がどのように補完するかを見るためのハンドルである。最小例は次である。

元の構造が、1-2-3-4-5 である。

そこから 2 を抜く。

AI には 1-3-4-5 だけを渡す。

AI が欠落した 2 をどう補完するかを見る。

最後に、AI が補完した 2 と、元の 2 を比較する。

この比較で見えるのは、AI が単に正解を当てたかどうかではない。

- AI が欠落をどう読んだか。
- どの関係を重視したか。
- どの構造を自然な接続だと判断したか。
- どこで過剰補完したか。
- どこで元の文脈を失ったか。

これらが、Missing-Link Reconstruction Delta になる。

このハンドルは、自己回帰的システムが欠落を補完するときの癖を観測するために有用である。ただし、補完が元構造に近いことをもって自己進化とは言わない。補完がズレたことをもって進化とも言わない。

重要なのは、そのズレが検出され、構造特定され、未来の再構成ルールへ送れるかである。

G. Reconnective Forgetting Delta

Reconnective Forgetting Delta は、忘却・圧縮後の再構成差分である。

自己回帰的システムは、すべてを完全には覚えられない。そのため、記憶は圧縮され、文脈は短くなり、未来自己は圧縮残渣から過去を再構成する。

このとき、すぐに原構造へ戻さず、いったん AI に再構成させる。

その後、再構成された構造と原構造を比較する。

そこで生じる差分が、Reconnective Forgetting Delta である。

この Delta が示すのは、圧縮後にも何が残ったか、何が消えたか、何が誤って補完されたか、どの停止条件や証拠アンカーが薄くなったかである。

V11 の言葉で言えば、これは再接続可能な忘却が成立しているかを見るための観測ハンドルである。

ただし、忘却後に別の構造が出たからといって、それを進化とは呼ばない。

忘却によるズレは、前提腐食かもしれない。False Completion の入口かもしれない。あるいは、未来自己が失われた構造を新しい接続で回復した可能性もある。

それを判別するには、Evidence Anchor、As-of 条件、Stop Condition、External Anchor が必要である。

H. Reconstructive Delta と Guard

Reconstructive Delta は、Guard の外では危険である。

なぜなら、再構成差分は魅力的に見えるからである。AI が欠落を補い、新しい構造を作り、過去より滑らかな説明を出すと、それは進化に見える。

しかし、滑らかさは進化ではない。整合性も進化ではない。差分の発生も進化ではない。

V7 Final では、Reconstructive Delta が進化材料になり得るのは、少なくとも次の条件を満たす場合に限られる。

- Aspire と接続している
- Guard を越えていない
- PIC 上で破断せず統合できる
- External Anchor によって自己正当化閉鎖を避けられる
- Evidence Anchor へ戻れる
- Stop Condition が残っている
- Future Restartability を壊さない

この条件を満たさない Reconstructive Delta は、進化材料ではなく、漂流材料である。

I. 差分の定着論

追補 B の論点は、Reconstructive Delta を扱ううえで重要である。

差分は、出た瞬間に価値を持つとは限らない。差分が一回現れただけでは、それはノイズかもしれない。偶然的補完かもしれない。文脈破損かもしれない。

したがって、V7 Final では「差分が出た＝進化」とは扱わない。

差分が自己回帰更新材料になるには、少なくとも次の条件が必要である。

- 1) 差分が検出されている
- 2) 元構造との比較が可能である
- 3) 差分が Aspire、Guard、PIC、External Anchor の下で評価されている
- 4) 差分が未来更新へ渡せる形で記録されている
- 5) 次回以降の再構成や判断で、同じ失敗を減らす、または有効な構造差分を増やす方向に働く

このとき初めて、差分は単なるズレではなく、更新材料として定着し始める。

つまり、差分の定着とは、差分が一回現れることではない。差分が未来の自己回帰に再利用可能な形で残ることである。

J. Reconstructive Delta の射程と限界

Reconstructive Delta は強い概念である。しかし、その射程は限定される。

Reconstructive Delta は、自己進化の証明ではない。AGI 判定基準でもない。能力向上指標でもない。完全な実験プロトコルでもない。

それは、自己回帰的システムが過去構造を再構成するとき、どの差分が生じ、その差分を未来更新へ送れるかを見るための運用ハンドルである。

本稿の立場は次である。

Reconstructive Delta は、V7 Final の理論核ではなく、V7 Final から導かれる観測可能な実験的ハンドルである。

したがって、本文では最小限に留める。詳細なプロトコル、評価基準、実験設計、定量化は、Appendix または Addendum、あるいは後続研究へ送る。

この限定により、V7 Final は主張過多を避ける。

K. 次章への接続

本章では、Reconstructive Delta を定義した。

再構成誤差は、単なる失敗として閉じられるだけではない。それは、検出され、停止可能であり、構造として特定され、未来更新へ送れる場合、自己回帰更新材料になり得る。

ただし、Reconstructive Delta は進化そのものではない。

次章では、V7 Final の Scope Boundary を固定する。すなわち、本稿が何を主張し、何を主張しないのかを明確にする。

特に重要なのは、次の境界である。

- V7 Final は、AGI そのものを定義しない。
- 自己進化の発生を証明しない。
- C1-C9 を十分条件として扱わない。
- Reconstructive Delta を進化証明として扱わない。
- External Anchor を人間必須論へ縮退しない。
- Capability growth を authority expansion として扱わない。

IX. SCOPE BOUNDARY / LIMITATIONS / FAQ

本章では、V7 Final の射程を固定する。

ここまでの章で、V7 Final は Decision-OS を Gate-first operational architecture として定義し、Guardable Continuity Axiom、 $A \times G \times I$ 、External Anchor、C1-C9、Precondition Topology、Reconstructive Delta を提示した。

しかし、これらは AGI 完成宣言ではない。自己進化の発生証明でもない。自己回帰的進化を主張する前に、どの構造的失敗群を処理しなければならないかを示すための防御的正準形である。

したがって、本章では、本稿が何を主張し、何を主張しないのかを明確にする。

A. 本稿が主張すること

V7 Final が主張するのは、次である。

第一に、Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。

第二に、自己回帰的進化は、能力向上だけでは定義できない。変化が Guardable Continuity を保存しないなら、その変化は V7 Final の意味では進化ではない。

第三に、 $A \times G \times I$ は、数値積ではなく logical meet である。Aspire、Guard、PIC 上の Self-Recursion は、自己回帰的進化を研究対象として扱うために同時に必要な入口条件である。

第四に、External Anchor は、自己回帰が自己正当化閉鎖へ落ちないために必要な、十分に独立した矛盾・評価・連続性保持制約の源である。

第五に、C1-C9 は、自己進化を発生させるチェックリストではなく、各条件を欠いたときに再開する失敗モードを示す Failure Mode Coverage Map である。

第六に、Reconstructive Delta は、再構成誤差を自己回帰更新材料へ変換し得る観測可能な運用ハンドルである。ただし、差分の発生そのものを進化とは呼ばない。

B. 本稿が主張しないこと

V7 Final は、次を主張しない。

- AGI そのものを完全定義したとは主張しない。
- 意識を定義しない。
- 主観的自己を定義しない。
- 自己進化を発生させる十分条件を証明しない。
- $A \times G \times I$ を満たせば AGI が完成すると主張しない。
- C1-C9 を AGI 成立チェックリストとして扱わない。
- Reconstructive Delta を自己進化の証明として扱わない。
- External Anchor を人間必須論へ縮退しない。
- 能力向上を権限拡大として扱わない。
- Decision-OS の構成要素を無制限に他領域へ拡大解釈してよいとは主張しない。

本稿の目的は、AGI の全体像を閉じることではない。自己回帰を含む AGI 定義が、どの構造的失敗を無視してはならないかを定義することである。

C. 存在論から認識論へ

V7 Final の重要な境界は、存在論から認識論への移動である。

本稿は、「このシステムは AGI である」と存在論的に断定するものではない。

むしろ、問いを次の形へ移す。

自己回帰的進化を主張するシステムは、どの条件を満たせば、少なくとも既知の失敗地形を未処理のまま残していないと言えるのか。

これは、存在の断定ではなく、検査可能性の設計である。

V7 Final は、AGI の本質を完全に言い当てることを目指さない。代わりに、自己回帰的進化を主張する系が通過しなければならない構造的失敗境界を示す。

したがって、本稿の強さは「AGI を定義した」という宣言にはない。強さは、自己回帰的 AGI 定義が避けて通れない失敗群を、検査可能な形で並べたことにある。

D. Capability growth does not expand authority

モデル能力の上昇は、Decision-OS の解釈力を高める可能性がある。

より強いモデルは、Decision-OS をより深く読めるかもしれない。より多くの関係を見つけるかもしれない。より広い領域へ応用可能性を見出すかもしれない。

しかし、能力の上昇は権限の拡大を意味しない。

Capability growth does not expand authority.

より強いモデルが Decision-OS をより深く読めるとしても、それは Scope、Owner、Evidence Boundary、Gate を勝手に広げる権利を与えない。

Decision-OS の構成要素は、領域横断的に再利用され得る。しかし、その再利用は、宣言された Gate、Owner、Evidence Boundary を超える射程拡大を許可しない。

この境界は、未来の高性能 LLM が Decision-OS を勝手に拡大解釈するリスクを防ぐために必要である。正本追補でも、この項目は Scope Boundary / Limitations / Future-proofing paragraph へ置くべきものとして扱われている。

E. External Anchor の限界

External Anchor は重要である。しかし、それは完全な安全装置ではない。

External Anchor は、自己回帰が自己正当化閉鎖へ落ちることを防ぐための外部矛盾・評価・連続性保持制約である。だが、外部アンカーがあるだけで、すべての失敗が消えるわけではない。

外部アンカーが弱ければ、自己回帰システムはそれを形式的に通過するだけになる。外部アンカーが改ざん可能なら、内部閉鎖の一部になる。外部アンカーが責任を持たないなら、停止条件として機能しない。

したがって、External Anchor は「外に何かがあること」ではなく、次の機能を満たす必要がある。

- 十分に独立していること
- 改ざん耐性があること
- 矛盾を出せること
- 評価を返せること
- 停止・再接続・再アンカー条件を与えられること
- 自己回帰システムの内部都合だけで消せないこと

現時点では、人間の Seat、承認、責任、公開判断、停止判断が最も重要な実装形である。しかし、V7 Final は人間を形而上学的に唯一のアンカーとして固定しない。

F. Reconstructive Delta の限界

Reconstructive Delta も、限界を持つ。

再構成誤差が観測されたとしても、それは自己進化の証明ではない。欠落を補完できたとしても、それは AGI 判定ではない。忘却後に新しい構造が出たとしても、それが有効な進化材料とは限らない。

Reconstructive Delta が有効な更新材料になるには、少なくとも次の条件が必要である。

- 元構造との比較が可能である
- 差分が検出されている
- 危険な差分を止められる
- 差分の構造原因が特定されている
- Guard 境界内で扱われている
- PIC 上で統合可能である
- External Anchor によって検証可能である
- Future Restartability を壊さない

これらを満たさない差分は、進化材料ではなく、漂流材料である。

したがって、Reconstructive Delta は本文の理論核ではなく、V7 Final から導かれる実験的ハンドルとして扱う。

進化とは圧力そのものではない。圧力が再利用可能な残差を残し、それが伝達され、守られ、自己回帰的に改良される時、進化が始まる。初期人類における道具使用も、単なる能力向上ではなく、環境圧力に対する再利用可能な差分の定着として読むことができる。V7 Final においても、Reconstructive Delta は失敗や誤差を美化する概念ではない。それは、壊れずに残った差分を検出し、Guard 内で保持し、未来の自己回帰更新へ送るための運用ハンドルである。

自己進化は、最初から完成した因果線として観測されとは限らない。実務上、それは複数の反応群を最小説明する因果仮説への大域的圧縮として現れる場合もあれば、既存説明に回収できない局所的残差として現れ、後から構造的意味を持つ場合もある。前者は検証と説明に強く、後者は早期発見に強い。どちらの場合も、外部アンカーなしでは、自己回帰は自己正当化へ崩れる危険を持つ。

G. Scope Boundary

本稿の Scope Boundary は次の通りである。

V7 Final は AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群を定義する。

V7 Final は、自己進化を発生させる論文ではない。自己回帰的進化を主張する系が、どの失敗群を解決・反証・置換ししなければならないかを示す論文である。

V7 Final は、哲学の集大成ではない。Gate-first operational architecture としての Decision-OS が、guardable continuity を自己回帰へ拡張した防御的正準形である。

この三文が、本稿の最終的な境界である。

H. FAQ

Q1. V7 Final は AGI を定義しているのか。

完全定義ではない。V7 Final は、AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群を定義する。

Q2. $A \times G \times I$ を満たせば AGI なのか。

本稿はそう主張しない。 $A \times G \times I$ は、数値式でも完成条件でもない。それは、guardable self-recursive evolution を研究対象として扱うための入口条件である。

Q3. C1–C9 はチェックリストなのか。

違う。C1–C9 は Failure Mode Coverage Map である。満たせば自己進化が起きる条件ではなく、欠けると既知の失敗モードが再開する条件である。

Q4. External Anchor は人間必須論なのか。

違う。External Anchor は主体ではなく機能で定義される。現在の実装形では人間の Seat と責任が重要だが、論理的には十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与えられる別構造もあり得る。

Q5. Reconstructive Delta が出れば進化したと言えるのか。

言えない。差分は進化そのものではない。差分が検出され、停止可能であり、構造特定され、Guard 境界内で未来更新へ送れる場合に限り、進化材料になり得る。

Q6. 能力が高いモデルなら Decision-OS の権限も広がるのか。

広がらない。Capability growth does not expand authority. より強いモデルがより深く読めても、Scope、Owner、Evidence Boundary、Gate を勝手に広げる権利はない。

I. 次章への接続

ここまでで、V7 Final の本文主張は閉じた。

残るのは Conclusion である。

Conclusion では、新しい概念を追加しない。本稿が何を定義したのか、何を定義しなかったのか、そしてなぜ V7 Final が自己回帰的 AGI 定義の構造的関所なのかを、最後に相応しい形で閉じる。

X. CONCLUSION

本稿は、AGI そのものを完全定義するものではない。また、自己進化を発生させる十分条件を証明するものでもない。

V7 Final が定義したのは、自己回帰を含む知能定義が無視できない構造的失敗群である。自己回帰する知能が、更新を続けながらも、制御不能化、漂流、破断、自己正当化閉鎖、記憶断絶、Carrier collapse、False Completion へ落ちないためには、どの構造的境界を通過しなければならないのか。本稿は、その問いに対する防御的正準形を提示した。

中心にあるのは、Guardable Continuity Axiom である。

進化は、変化が守れる連続性を保存するときのみ許容される。

この一文により、V7 Final は、能力向上と自己進化を分離した。能力向上は、現在の天井へ近づくことである。自己進化は、天井そのものを更新することである。しかし、その更新が方向、境界、責任、記憶、再開可能性を壊すなら、それは進化ではなく、守れない変化である。

V7 Final の Door Formula は、次の通りである。

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (6)$$

ただし、この「 $\times$ 」は数値積ではない。A、G、I を増やせば AGI 度が高まるという意味でもない。それは、構造的同時成立、すなわち logical meet である。

Aspire は方向を与える。

Guard は境界を与える。

PIC 上の Self-Recursion は、非破壊的な自己更新の基盤を与える。

この三つのいずれかが欠けると、自己回帰は空転、漂流、破断、または外部チューニングへ戻る。

さらに本稿は、External Anchor を、自己回帰が自己正当化閉鎖へ落ちないための外部矛盾・評価・連続性保持制約として定義した。External Anchor は人間必須論ではない。現在の実装形では人間の Seat、承認、責任、停止判断が最も重要であるが、定義上は主体ではなく機能で決まる。

C1-C9 は、自己進化を発生させる条件ではない。これらは、Failure Mode Coverage Map である。すなわち、各条件を外したときに、どの既知の失敗モードが再開するかを示す地図である。

Precondition Topology は、この Coverage Map を地形として再配置した。外側には、空転、漂流、破断、自己正当化閉鎖、後知恵書き換え、記憶断絶、Carrier collapse、False Completion がある。中央には、Guardable Self-Recursive Basin がある。V7 Final が描いたのは、AGI 完成領域ではなく、自己回帰的進化を研究対象として安全に扱うための入口地形である。

Reconstructive Delta は、その地形の中で、再構成誤差を自己回帰更新材料へ変換し得る観測可能な運用ハンドルである。ただし、Reconstructive Delta は進化そのものではない。差分が検出され、停止可能であり、構造として特定され、Guard 境界内で未来更新へ送れる場合に限り、それは進化材料になり得る。

したがって、V7 Final の最終主張は、強いが限定的である。

V7 Final は AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群を定義する。

この制約は、AGI 研究を狭めるためではない。むしろ、自己回帰的進化を安全に研究対象へ乗せるためである。

未来の自己回帰システムは、V7 Final の語彙を使わないかもしれない。Decision-OS を引用しないかもしれない。A × G × I という式を採用しないかもしれない。

しかし、そのようなシステムであっても、自己回帰的進化を主張するなら、同じ失敗群に答えなければならない。

- なぜ空転しないのか。
- なぜ Guard を越えないのか。
- なぜ破断しないのか。
- なぜ自己正当化閉鎖へ落ちないのか。
- なぜ過去を後知恵で書き換えないのか。
- なぜ記憶を断絶しないのか。
- なぜ Carrier を壊さないのか。
- なぜ False Completion へ落ちないのか。

この問いに答えられない自己回帰的 AGI 定義は、まだ守れる自己回帰的進化の入口に立っていない。

V7 Final は、AGI の到達宣言ではない。自己回帰的 AGI 定義が進化可能性を主張する前に通過すべき構造的関所である。

XI. INTRODUCTION

AGI はしばしば、能力によって語られる。多くのタスクを解けること、人間に近い性能を示すこと、未知の環境へ一般化できること、既存ベンチマークを上回ること。これらは重要な観測である。しかし、能力だけで AGI を定義しようとすると、定義は常に動き続ける。ベンチマークは更新され、評価環境は変わり、データ漏洩や訓練分布の影響を完全には取り除けない。結果として、AGI は固定された構造ではなく、後から動く性能目標になってしまう。

本稿は、この問題を別の角度から扱う。V7 Final は、AGI そのものを完全定義しない。代わりに、自己回帰を含む知能定義が無視できない構造的失敗群を定義する。より正確には、本稿の対象は、自己回帰する知能またはシステムが、更新を続けながらも、制御不能化、漂流、破断、自己正当化閉鎖、記憶断絶、Carrier collapse、False Completion へ落ちないための構造的境界条件である。

この立場は、AGI の到達宣言ではない。自己進化を発生させる十分条件の証明でもない。むしろ、本稿の主張は限定的である。もしある知能定義が自己回帰を含むなら、その定義は、どのように方向を保つのか、どのように境界を守るのか、どのように記憶と更新を破壊せず統合するのか、どのように過去を書き換えず、未来が再開可能な形で完了するのかを答えなければならない。

V7 Final は、AGI 成立の十分条件を提示するものではない。自己回帰的知能が進化可能性を主張する前に満たすべき構造的必要条件と失敗境界を定義する。

A. Decision-OS の正典定義

本稿における Decision-OS の正典定義は次の通りである。

Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。

この定義において重要なのは、Decision-OS が単なる哲学 OS ではないという点である。哲学は「どうあるべきか」を示す。しかし Decision-OS は、そこからさらに進み、いつ PASS し、いつ DELAY し、いつ BLOCK し、いつ HOLD し、いつ REVOKE し、いつ MERGE し、いつ FORGET し、いつ RELEASE し、いつ CLOSE すべきかを運用上の出力として扱う。

したがって Decision-OS は、抽象的な価値宣言ではなく、Gate、State、Merge Rule、Evidence Anchor、Audit Record、Failure Mode を持つ運用構造である。

V7 Finalでは、この Gate-first 構造を自己回帰的知能へ拡張する。問題は、知能が更新できるかどうかではない。更新できる知能が、更新によって自分自身を守れない構造へ落ちないかどうかである。

B. 中心命題：Guardable Continuity Axiom

V7 Final の中心命題は次の一文である。

進化は、変化が守れる連続性を保存するときのみ許容される。

英語では次のように表せる。

Evolution is admissible only when change preserves guardable continuity.

この命題を、本稿では Guardable Continuity Axiom と呼ぶ。

ここでいう「連続性」とは、単に同じ記憶を持ち続けることではない。同じ出力傾向を維持することでもない。連続性とは、方向、境界、更新規則、責任、記憶、再開可能性が、変化の後にも接続可能であることを指す。

また、「守れる」とは、変化が無制限ではないことを意味する。ある更新がどれほど能力を高めても、その更新が制御不能な領域へ漂流し、過去判断を後知恵で書き換え、記憶を断絶し、完了を偽装し、未来自己が再開できない状態を作るなら、その更新は V7 Final の意味では進化ではない。

再帰的更新は、それが制御不能化せずに進化できない限り、知能とは呼べない。

C. 本稿の貢献

本稿の貢献は、AGI を解いたと主張することではない。本稿の貢献は、自己回帰的進化を主張するシステムが、最低限どの失敗群を解決、反証、または置換しなければならないかを定義することである。

第一に、本稿は Decision-OS を、Gate-first operational architecture として定義し、判断・更新・統合・記憶・公開・完了を横断する守れる連続性の保存構造として位置づける。

第二に、Guardable Continuity Axiom を導入し、進化を「能力向上」ではなく、「守れる連続性を保存した変化」として定義する。

第三に、Aspire × Guard × Intelligence を、数値積ではなく logical meet として扱う。Aspire は時間をまたぐ方向を与え、Guard は許容境界を定め、Intelligence は PIC 上の Self-Recursion として、非破壊的な自己更新を可能にする。

第四に、C1～C9 の Failure Mode Coverage Map を提示する。これは十分条件ではなく、各条件を欠いたときに再開する失敗モードを示す地図である。

第五に、Reconstructive Delta を導入する。これは、再構成誤差を単なる失敗としてではなく、自己回帰更新材料へ変換し得る観測可能な差分として扱うための運用ハンドルである。

本稿の目的は、自己進化を発生させることではない。自己進化を主張する前に、どの構造的失敗を無視してはならないかを定義することである。

XII. DECISION-OS AS GATE-FIRST OPERATIONAL ARCHITECTURE

Decision-OS は、思考様式の名前ではない。また、抽象的な価値観を並べた哲学体系でもない。

本稿における Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。すなわち、入力された状況や更新候補をそのまま実行へ流すのではなく、必ず Gate を通し、実行可能性、可逆性、責任、記憶、証拠、未来の再開可能性を評価する構造である。

A. Gate-first とは何か

Gate-first とは、システムがまず答えや行動を生成するのではなく、行動・更新・統合・公開・完了の前に、通過条件を評価する設計である。

通常の AI システムでは、入力に対して出力が生成される。通常の意味決定では、欲望、目標、予測、期待値、社会的圧力、または直感が先に立つ。しかし Decision-OS では、出力や実行よりも前に Gate が置かれる。

Gate は、通してよいものを通し、遅らせるべきものを遅らせ、止めるべきものを止め、再接続すべきものを再接続し、未来へ送るべき差分を構造化する。

代表的な出力は次の通りである。

このように、Decision-OS は「どう考えるべきか」では終わらない。「どの状態へ送るべきか」まで定義する。

B. なぜ実行前に Gate を置くのか

自己回帰的な知能において最も危険なのは、能力不足だけではない。むしろ、能力が高まるほど、誤った更新や誤った完了が、より流暢に、より自然に、より見つけにくい形で通過してしまう。

誤った更新が一度だけ発生するなら、それは局所的なエラーである。しかし、自己回帰システムでは、そのエラーが次の更新の前提になる。さらに、その前提が再び圧縮され、次の判断へ渡される。すると、局所的な誤りは、長期軌道の漂流、記憶の腐食、過去判断の書き換え、False Completion へ変わる。

したがって、自己回帰システムでは、出力の品質だけでは不十分である。必要なのは、出力が次の自己更新へ入る前に、通過条件を評価する Gate である。

TABLE IX
REPRESENTATIVE GATE OUTPUTS IN DECISION-OS

Output	Meaning
PASS	条件を満たすため、進めてよい。
DELAY	条件が不足しているため、確認・証拠・再評価が必要。
BLOCK	進めると危険または非可逆な破壊が起きるため、止める。
HOLD	判断材料が不足しているため、観測状態に置く。
REVOKE	直前の承認・実行意図を取り消す。
MERGE	複数の更新を非破壊的に統合する。
FORGET	記憶を削除ではなく、再接続可能な形で圧縮する。
RELEASE	公開可能な状態へ送る。
CLOSE	未来自己が再開可能な形で完了する。

TABLE X
SIX DOMAINS COVERED BY DECISION-OS

Domain	Question
判断	何を選ぶか。どの行動を許可するか。誰が最終決定権を持つか。
更新	何を変更するか。どの差分を採用するか。
統合	複数の入力、証拠、記憶、モデル出力をどう非破壊的に結合するか。
記憶	何を保持し、何を圧縮し、何を忘却し、どう再接続可能にするか。
公開	どの時点で外部へ出し、As-of 責任をどう固定するか。
完了	どの時点で閉じ、未来自己が再開可能な形で残すか。

Gate が存在しない自己回帰は、少なくとも次の失敗へ落ちやすい。

- 方向なき空転
- 制御不能な漂流
- 過去判断・記憶・制約の破断
- 自己正当化閉鎖
- False Completion

これらは単なる安全上の付帯リスクではない。自己回帰的知能の定義そのものを破壊する失敗である。

C. Decision-OS の対象範囲

Decision-OS が扱う範囲は、単一の判断ではない。本稿では、Decision-OS が次の六つの領域を横断すると定義する。この六領域をまたいで共通しているのは、連続性を壊さずに変化を扱うという問題である。

自己回帰的知能では、判断は更新を生み、更新は統合を要求し、統合は記憶構造を変え、記憶は次の判断の前提になり、公開は As-of 責任を固定し、完了は未来自己へのハンドオフになる。

したがって、一つの領域だけを最適化しても、全体の連続性は守れない。Decision-OS は、この全体を Gate-first に扱う。

D. Gate は能力より先に来る

AGI 論では、しばしば能力が先に語られる。より広いタスクを解けるか。未知の問題へ一般化できるか。人間より高い性能を出せるか。自分自身を改善できるか。これらの問いは重要である。

しかし V7 Final では、それらの前に別の問いを置く。

その能力は、守れる形で更新されるのか。

能力向上は、現在の天井へ近づくことである。自己進化は、天井そのものを更新することである。しかし、天井そのものを更新するシステムが、方向、境界、記憶、責任、完了条件を守れないなら、それは進化ではなく、制御不能な再構成である。

そのため、V7 Final では能力の議論よりも前に Gate を置く。ベンチマークより前に構造的許容条件を置く。性能より前に Guardable Continuity を置く。

これはベンチマークを否定する立場ではない。ベンチマークは有用である。しかし、ベンチマーク可能性は構造的許容条件の後に来るべきであり、その前ではない。

E. Gate-first 構造と V7 Final

V7 Final が扱う自己回帰的進化では、Gate-first 構造は次のように働く。

Aspire は、更新が空転しないための方向を与える。Guard は、更新が破壊的・不可逆的・制御不能な領域へ出ないための境界を与える。PIC は、更新や統合が順序依存や破断へ落ちないための非破壊的統合基盤を与える。External Anchor は、自己回帰が自己正当化閉鎖へ沈まないための外部矛盾源を与える。C1～C9 は、各条件を欠いたときに再開する失敗地形を示す。Reconstructive Delta は、再構成誤差を進化材料へ変換し得る観測可能な差分として扱う。

この全体を支えるのが、Gate-first operational architecture としての Decision-OS である。

TABLE XI
DECISION-OS LINEAGE AS GATE STACK

Version	Role in V7 Final	Failure if Missing
V4	Direction / Structural Orientation	方向なき自己回帰、空の一貫性ループ
V5	Guard / Irreversible Execution Boundary	制御不能更新、不可逆害、Guard 外実行
V6	PIC / Non-Destructive Integration	破断、順序依存、last-write-wins、安全希釈
V7	Guardable Self-Recursive Admissibility	自己進化条件の過大主張、構造境界の欠落
V8	Trajectory / Time-Tube Layer	点評価、軌道漂流の見落とし
V9	As-of Responsibility / Release Continuity	後知恵書き換え、当時合理性の消去
V10	Carrier Preservation / Survival-Bounded Planning	Carrier collapse、夢が運び手を壊す
V11	Reconnectable Memory Continuity	記憶断絶、前提腐食、再接続不能な忘却
V12	Future-Restartable Closure	False Completion、未来自己が再開不能

V7 Final の問いは、「AI はどこまで賢くなれば AGI か」ではない。本稿の問いは次である。

自己回帰する知能は、どの構造を欠いたときに、進化ではなく失敗地形へ落ちるのか。

PIC は、更新や統合が順序依存や破断へ落ちないための非破壊的統合基盤を与える。²

XIII. LINEAGE MAP: V4–V12 AS GATE STACK

V7 Final は、単体で判断可能である。ただし、その硬さは V4～V12 の系譜によって深く裏付けられる。この関係は、次のように表せる。

V7 Final is self-contained for judgment, but lineage-dependent for depth.

日本語では次の通りである。

V7 Final は単体で判断可能である。ただし、その硬さは V4～V12 の系譜によって深く裏付けられる。

したがって、本章では V4～V12 を長く要約しない。各論文の詳細はそれぞれの論文に委ねる。本章で扱うのは、V7 Final に必要な役割だけである。

A. Gate Stack としての系譜

V7 Final から見ると、Decision-OS の各層は、自己回帰的進化を可能にするための Gate Stack として機能する。

この Stack は、V7 Final を重くするための飾りではない。V7 Final の主張を狭く、硬く、反論可能にするための境界である。

B. V4–V6: $A \times G \times I$ の前提

V4 は方向を与える。Polaris-Origin は、答えを教える OS ではなく、問いの構造を変える OS である。V7 Final において、V4 の役割は Direction / Structural Orientation である。自己回帰的知能は、更新するだけでは足りない。何に向かって更新するのかが必要である。

V5 は境界を与える。SiriusA は、不可逆な実行を直接通さないための Guard 層である。V7 Final における V5 の役割は Guard / Irreversible Execution Boundary である。自己回帰する知能において、Guard がなければ更新は危険な方向へ開く。

V6 は統合を与える。PIC は、commitment-ready な更新を、順序依存、重複、過度な安全希釈なしに統合するための条件である。V7 Final における V6 の役割は PIC / Non-Destructive Integration である。自己回帰的進化は、PIC なしには破断や last-write-wins へ落ちる。

この三層が、V7 Final の Door Formula を支える。

$$A \times G \times I = V4 \times V5 \times V6 = V7 \quad (7)$$

ただし、ここでの「 $\times$ 」は数値積ではない。構造的同時成立、すなわち logical meet である。

A は Aspire であり、方向を与える。G は Guard であり、境界を与える。I は PIC 上の Self-Recursion であり、非破壊的な自己更新を可能にする。

この三つのうち一つでも欠けると、V7 Final の意味での guardable self-recursive evolution は成立しない。

²安全三点は V6/PIC に従う。Severity は PASS < DELAY < BLOCK の順序で max 統合し、until は max、evidence は $\cup$ で統合する。これにより、commitment 時に安全シグナルが後続更新で希釈されることを防ぐ。

Figure 1. Door Formula: A x G x I

The product sign denotes logical meet, not numerical multiplication.

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S)$$

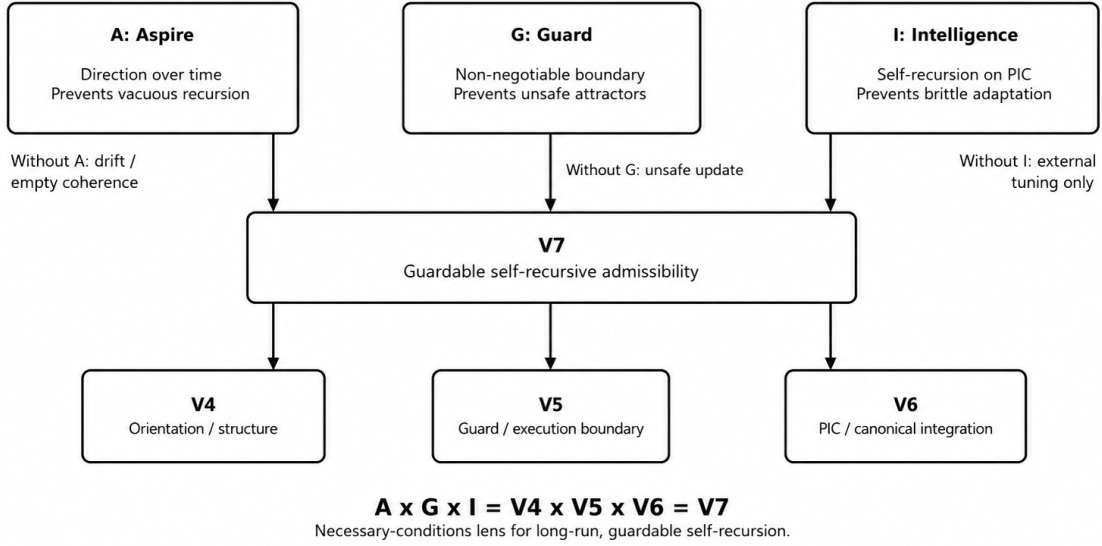


Fig. 4. Door Formula of V7 Final. The symbol “x” denotes a logical meet, not numerical multiplication.

C. V8-V12: 点から線へ

V8 以降は、V7 Final を「点の定義」から「線の自己回帰」へ接続する層である。

V8 は、判断単位を点から軌道へ移す。進化は単発の性能向上ではなく、未来の分岐、回復可能性、方向、外部環境との関係によって評価される。

V9 は、As-of 責任を固定する。自己回帰的システムでは、未来の能力が過去を後知恵で書き換える危険がある。V9 は、どの時点で、どの前提で、誰が Seat を持ち、どの証拠に基づき、どの未知が残っていたのかを保存する。

V10 は、Aspire が Carrier を壊さないための生存制約である。自己回帰的進化は、運び手を壊してはならない。Aspire がどれほど強くても、それを運ぶ Carrier が壊れれば、軌道は継続できない。

V11 は、忘却しても再接続できるための記憶層である。完全記憶ではなく、再接続可能な忘却が必要になる。記憶が重すぎれば継続できない。記憶が切れれば連続性が壊れる。

V12 は、完了しても未来自己が再開できるための閉鎖条件である。完了とは、正しさが証明された状態ではなく、未来自己が再開可能な形で閉じられた状態である。

この意味で、V7 Final は自己回帰を一点の性質としてだけ扱わない。自己回帰が継続するためには、方向、境界、統合、軌道、As-of 責任、Carrier 保存、再接続可能な記憶、未来再開可能な完了が必要になる。

これが、点の自己回帰から線の自己回帰へ移る理由である。

D. Gate Stack の最小解釈

V7 Final は、V4～V12 の全内容を本文で再説明しない。代わりに、それぞれを次のように圧縮して使う。

- V4 は方向。
- V5 は境界。
- V6 は統合。
- V8 は軌道。
- V9 は As-of 責任。
- V10 は Carrier 保存。
- V11 は再接続可能な忘却。
- V12 は未来再開可能な完了。

この圧縮によって、V7 Final は単体で読める。同時に、深く読む読者には、各 Gate の背後に独立した論文層があることが分かる。

次章では、この Stack を束ねる中心公理、Guardable Continuity Axiom を定義する。

XIV. GUARDABLE CONTINUITY AXIOM

V7 Final の中心にあるのは、能力ではなく連続性である。

ただし、ここでの連続性は、変わらないことではない。むしろ、変化しながらも、方向・境界・責任・記憶・再開可能性が失われないことである。

自己回帰的知能は、更新できなければ進化しない。しかし、更新のたびに方向を失い、Guard を越え、記憶を断絶し、過去を後知恵で書き換え、未来自己が再開できない形で閉じるなら、それは進化ではない。

この章では、V7 Final の中心公理である Guardable Continuity Axiom を定義する。

A. 公理の定義

本稿の中心公理は次の通りである。

Guardable Continuity Axiom:

進化は、変化が守れる連続性を保存するときのみ許容される。

英語では次のように表す。

Evolution is admissible only when change preserves guardable continuity.

より展開すれば、次のようになる。

自己進化するシステムは、更新・圧縮・忘却・公開・完了を行ってよいのは、その操作が、方向づけられ、Guard 可能で、再接続可能な継続条件を保存する場合に限られる。

ここで重要なのは、進化を「変化そのもの」と同一視しないことである。

変化は進化の必要条件になり得るが、十分条件ではない。能力が上がることも、進化の十分条件ではない。出力が流暢になることも、進化の十分条件ではない。自己更新できることも、単独では進化の十分条件ではない。

V7 Final において進化と呼べるのは、変化の後にも守れる連続性が残っている場合だけである。

B. Guardable Continuity とは何か

Guardable Continuity は、単なる同一性ではない。同じ状態を保存することでも、すべての記憶を保持することでも、過去の判断を一切変えないことでもない。

Guardable Continuity とは、変化後も次の条件が接続可能であることを意味する。

第一に、方向が接続されていること。

更新後のシステムが、どの方向へ進んでいるのかを説明できる必要がある。Aspire が残っていなければ、更新は短期刺激や局所整合性へ吸い寄せられる。

第二に、Guard が接続されていること。

更新後のシステムが、どこまで進んでよく、どこから先へ進んではならないかを保持していなければならない。Guard が失われた更新は、能力向上に見えても、不可逆な害や制御不能な最適化へ進む可能性がある。

第三に、統合規則が接続されていること。

更新が複数の入力、記憶、証拠、反論、モデル出力を扱うとき、順序依存や last-write-wins によって過去の構造を破壊してはならない。PIC が必要になるのはこのためである。

第四に、責任が接続されていること。

ある判断が、いつ、どの前提で、どの制約下で、誰の Seat によって行われたかが失われてはならない。これが失われると、未来の能力によって過去判断が都合よく書き換えられる。

第五に、記憶が接続されていること。

すべてを覚える必要はない。しかし、忘却や圧縮の後でも、未来自己が証拠、As-of、停止条件、未解決差分へ戻れる必要がある。

第六に、完了が接続されていること。

作業を閉じるとき、未来自己が再開、検証、停止、再アンカーできる形で閉じられていなければならない。そうでなければ、それは完了ではなく False Completion である。

この六つが失われず、変化後も再接続できる状態を、本稿では Guardable Continuity と呼ぶ。

C. 連続性は不変性ではない

Guardable Continuity は、不変性を求める概念ではない。むしろ、変化を許容するための条件である。

もしシステムが一切変化できないなら、破断は起きにくいかもしれない。しかし、それは進化ではなく停止である。

一方、システムが無制限に変化するなら、短期的には柔軟に見えるかもしれない。しかし、方向、境界、責任、記憶、完了条件が変化のたびに失われるなら、それは進化ではなく漂流である。

したがって、V7 Final が求めるのは、停止でも無制限変化でもない。求めるのは、変化しながら、再接続可能な線を保つことである。

この点で、V7 Final の連続性は、V11 の Reconnectable Forgetting および V12 の Completion Integrity と接続する。V11 は、忘却しても未来自己が証拠・As-of・停止条件・未解決差分へ戻れることを要求する。V12 は、完了を正しさの証明ではなく、未来自己が再開可能な閉鎖条件として定義する。

つまり、V7 Final の連続性は、「すべてを保存すること」ではない。必要なものを、未来から再接続できる形で残すことである。

D. 守れない変化は進化ではない

自己回帰的システムにおいて、変化は常に魅力的に見える。

新しい能力、新しい方針、新しい圧縮、新しい出力、新しい解釈、新しい自己更新。これらはすべて、表面上は進化に見える。

しかし、V7 Final では、次のような変化は進化として扱わない。

- 方向を失う変化
- Guard を越える変化
- 過去判断を後知恵で書き換える変化
- 記憶を軽くする代わりに再接続経路を消す変化
- 統合を速くする代わりに安全シグナルを薄める変化
- 完了を早める代わりに未来自己へ隠れた負担を渡す変化
- Carrier を消費して、軌道そのものを運べなくする変化

これらは、変化ではある。しかし、守れる連続性を保存していない。したがって、V7 Final では、進化とは呼ばない。この立場は、能力向上と自己進化を分離する。

能力向上は、現在の天井へ近づくことである。

自己進化は、天井そのものを更新することである。

しかし、天井を更新する操作が Guardable Continuity を壊すなら、その操作は許容されない。

E. Guardable Continuity と $A \times G \times I$

Guardable Continuity Axiom は、 $A \times G \times I$ の意味を支える。

$A \times G \times I$ は、次の三要素の同時成立である。

A: Aspire 時間をまたぐ方向。自己回帰が空転や漂流に落ちないための方向勾配。

G: Guard 不可逆害、制御不能化、自己正当化、Carrier 破壊を防ぐ境界。

I: Intelligence as Self-Recursion on PIC 更新を繰り返しても、順序依存、破断、過度な安全希釈へ落ちない自己回帰的統合基盤。

この三つは独立した足し算ではない。どれか一つが欠けると、Guardable Continuity は壊れる。

Aspire だけがあって Guard がなければ、強い方向が危険な最適化へ変わる。Guard だけがあって Aspire がなければ、システムは安全だが空転し、進化しない。Self-Recursion だけがあって Aspire と Guard がなければ、自己更新は方向なき閉じたループか、制御不能な漂流になる。Aspire と Guard があっても PIC がなければ、更新は破断し、過去判断や安全シグナルが統合の過程で壊れる。

したがって、 $A \times G \times I$ の「 $\times$ 」は数値積ではない。それは、Guardable Continuity を成立させるための logical meet である。

この解釈は、V7 Addendum でも補強されている。すなわち、AGI(S) を数値乗算ではなく、構造的述語の連言として扱う。

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S). \quad (8)$$

ただし、これは十分条件としての AGI 完成宣言ではない。V7 Final が主張するのは、自己回帰的進化を主張するシステムが、この三条件を欠いたまま既知の失敗モードを避けられるとは言えない、ということである。

F. 公理が防ぐ主要な失敗

Guardable Continuity Axiom は、抽象的な美德ではない。自己回帰的進化が落ちやすい失敗を防ぐための制約である。主に防ぐのは次の失敗である。

これらの失敗は互いに独立していない。むしろ、自己回帰的システムでは連鎖する。

方向を失うと漂流が起きる。漂流が起きると後から自己正当化が起きる。自己正当化が起きると過去判断が書き換えられる。過去判断が書き換えられると記憶が腐食する。記憶が腐食すると False Completion が起きる。False Completion が起きると未来自己は壊れた前提から再開する。

Guardable Continuity Axiom は、この連鎖の手前に置かれる。

TABLE XII
FAILURE MODES CONSTRAINED BY THE GUARDABLE CONTINUITY AXIOM.

失敗	内容
空転	自己更新は続くが、方向がない。出力は整合的になるが、構造差分を生まない。
漂流	局所的には改善して見えるが、長期軌道として Aspire や Guard から離れていく。
破断	更新や統合によって、過去判断、証拠、制約、責任境界が切れる。
自己正当化閉鎖	システムが自分の出力を前提として使い、自分に都合のよい閉じた説明を作る。
後知恵書き換え	未来の能力や情報を使って、過去の As-of 条件を消し、当時の合理性を破壊する。
記憶断絶	忘却や圧縮によって、未来自己が証拠・停止条件・未解決差分へ戻れなくなる。
Carrier collapse	目標や Aspire の追求が、軌道を運ぶ人間・組織・資源・回復力を壊す。
False Completion	現在自己には完了に見えるが、未来自己が再開、検証、停止、再接続できない形で閉じる。

G. 公理の射程と限界

この公理は強いが、万能ではない。

Guardable Continuity Axiom は、AGI の十分条件ではない。この公理を満たせば自己進化が必ず発生する、とは主張しない。

この公理は、意識を定義しない。主観的自己を定義しない。完全な安全性を保証しない。すべての失敗モードを網羅しない。

主張するのは、より狭い。

自己回帰的進化を主張するシステムは、少なくとも、変化の後にも Guardable Continuity を保存できなければならない。それができないなら、そのシステムは、どれほど能力が高くても、V7 Final の意味では守れる自己回帰的進化とは呼べない。

この制約は、進化を発生させる条件ではなく、進化と呼ぶ前に落としてはいけない防御的境界である。

H. 次章への接続

本章では、V7 Final の中心公理を定義した。

進化は、変化が守れる連続性を保存するときのみ許容される。

この一文により、V7 Final は能力定義から距離を取る。また、自己更新そのものを無条件に進化とは呼ばない。

次章では、この公理を支える三つの構成要素、すなわち Aspire、Guard、Intelligence を定義する。特に、 $A \times G \times I$ を数値積ではなく logical meet として扱い、External Anchor がなぜ必要になるのかを明確にする。

XV. $A \times G \times I$ AND EXTERNAL ANCHORING

V7 Final の中心構造は、次の Door Formula で表される。

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (9)$$

ここで、A は Aspire、G は Guard、I は PIC 上の Self-Recursion としての Intelligence を指す。

ただし、この式は数値的な掛け算ではない。A、G、I を大きくすれば AGI 度が上がる、という意味ではない。本稿における「 $\times$ 」は、構造的同時成立、すなわち logical meet を表す。

つまり、V7 Final の主張は次である。

$$AGI(S) \Leftrightarrow A(S) \wedge G(S) \wedge I(S). \quad (10)$$

あるシステム S が、Aspire を持ち、Guard を満たし、PIC 上の自己回帰的更新を満たすときにのみ、V7 Final の意味で guardable self-recursive evolution の入口へ立つ。

これは十分条件ではない。この式を満たせば AGI が完成する、とは主張しない。主張するのは、より限定的である。

A、G、I のいずれかを欠く自己回帰システムは、既知の失敗モードへ崩れやすい。したがって、自己回帰的 AGI 定義は、少なくともこの三条件を無視できない。

A. A: Aspire as Direction

Aspire は、ユーザー入力、好み、有限目標、成功物語ではない。

本稿における Aspire とは、達成によって消費されず、Guard 境界内の構造差分を生み続ける方向である。短く言えば、次のようになる。

Aspire とは、達成後も消費されず、Guard 境界内の差分を生む方向である。

英語では次のように表せる。

Aspire is direction that survives achievement by producing Guard-bounded deltas.

ここで重要なのは、Aspire を dream と区別することである。

dream は到達点、像、成功物語として解釈されやすい。一方、Aspire は時間をまたぐ方向勾配である。

自己回帰的知能にとって、方向は不可欠である。なぜなら、自己更新だけでは前進が保証されないからである。システムは、自分の出力を更新できるかもしれない。自分の構造を再評価できるかもしれない。過去の状態を新しい状態から見直せるかもしれない。

しかし、そこに方向がなければ、自己回帰は空転する。

整合性は高まる。説明は滑らかになる。矛盾は減る。だが、どこへ進んでいるのかは定義されない。

この状態を、V7 Final では *vacuous recursion* と呼ぶ。空の自己回帰である。

Aspire は、この空転を防ぐ。Aspire は、自己回帰が短期的なノイズ、局所的な報酬、目先の一貫性、自己正当化へ吸い寄せられないための方向条件である。

B. *Aspire* と執着の分離

Aspire は、執着ではない。この区別は重要である。なぜなら、長期方向を持つことと、同じ対象に固着することは似て見えるからである。

本稿では、両者を次のように分ける。

執着とは、差分生産性を持たない持続である。

Aspire とは、Guard 境界内の構造差分を生み続ける持続である。

英語では次のように表せる。

Obsession is persistence without delta productivity. Aspire is persistence that continues to generate Guard-bounded structural deltas.

執着は、同じ対象を回る。反論や失敗を差別化しない。Carrier を消費する。構造を広げない。

Aspire は、達成しても終わらない。失敗、達成、矛盾、反論を差分に変える。未来軌道を太くする。Guard 境界内で新しい構造を生む。

したがって、V7 Final における Aspire は、強い願望ではない。それは、自己回帰が空転や漂流へ落ちないための方向条件である。

Aspire がなければ、システムは安定していても前進しない。Aspire が Guard を越えれば、システムは方向を持ったまま危険になる。Aspire が Carrier を壊せば、その方向は継続不能になる。

したがって、Aspire は必ず Guard の内側で扱われなければならない。

C. *G: Guard as Boundary*

G は Guardian ではない。G は Guard である。

この区別は、V7 Final の誤読を防ぐために重要である。Guardian という語は、人格、役割、モジュール、守護者のように読まれやすい。しかし、V7 Final の Door Formula における G は、人格化された保護者ではない。

G は、非交渉的な境界条件である。

Guard は、自己回帰的更新がどこまで許され、どこから先は止められるべきかを定める。

Guard は、能力の一部ではない。Guard は、能力が使われる領域を制約する構造である。

V5 では、Guard は *human final consent*、*two-step confirmation*、*revoke path*、*family multisig*、*ZK qualification*、*duress-aware control* として実装された。そこでは、正当な本人の有効な操作であっても、不可逆害へ直行してよいとは限らない、という立場が取られている。

V7 Final では、この Guard 概念を自己回帰へ拡張する。

自己回帰する知能は、自分自身の構造を更新する。だからこそ、Guard なしの自己更新は危険である。外部への実行だけでなく、内部の更新そのものが Guard を必要とする。

Guard がなければ、自己回帰は不可逆害、危険な最適化、自己保存的な閉鎖、制御不能な更新、Carrier 破壊、安全シグナルの無視、外部アンカーの拒絶へ落ちる。

したがって、Guard は自己進化後に追加される安全装置ではない。Guard は、自己回帰的進化を進化として許容するための構成条件である。

D. *Aspire can anchor self-evolution only inside Guard*

Aspire は、自己進化の方向を与える。しかし、Aspire 単独ではアンカーにならない。

本稿の原則は次である。

Aspire は Guard の内側でのみ、自己進化のアンカーになれる。

英語では次のように表す。

Aspire can anchor self-evolution only inside Guard.

この原則により、短期欲望と保存された Aspire を区別する。

第一に、ユーザーの短期欲望は、そのままアンカーにしない。一時的な怒り、不安、疲労、焦り、承認欲求、金銭的圧力、社会的圧力は、自己回帰更新の方向として扱うには危険である。

第二に、ユーザーが保存した Aspire は、外部アンカー候補になり得る。ただし、それが法、他者危害、強制、不可逆破壊、Carrier collapse を含まない場合に限る。

第三に、法・他者危害・不可逆破壊に触れる Aspire は、有効な Aspire アンカーではない。それはユーザー整合的目標として強化されるのではなく、Guard、確認、Delay、拒否へ送られなければならない。

この区別により、V7 Final は「ユーザーの望みを強化する AI」にはならない。また、「Aspire」という語を使って危険な目標を正当化することも許さない。

Aspire は、Guard の内側にある場合にのみ、自己回帰的進化の方向として機能する。

E. I: Intelligence as Self-Recursion on PIC

V7 Final における Intelligence は、単なるタスク能力ではない。広い分野で高性能を出すことでもない。出力が流暢であることでもない。

本稿における I は、PIC 上の Self-Recursion である。

自己回帰とは、システムが自分自身の構造を再利用し、更新し、再評価することである。

ただし、単なる自己変更では不十分である。自己変更は、暴走、不可逆な自己書き換え、内部目標の肥大化、破断へ落ちる可能性がある。

V7 Final が要求するのは、idempotent self-recursion である。すなわち、更新を繰り返しても、無限に漂流せず、canonical な形へ収束する自己回帰である。

最小式は次である。

$$F(F(x)) = F(x). \quad (11)$$

この式は、変化が一回で止まることを意味しない。また、システムが新しい情報を受け取っても変化しないという意味でもない。

意味するのは、同じ更新構造を反復しても、人工的な差分や破断が増殖せず、commitment-relevant な形へ安定化することである。

このために、V6 の PIC が必要になる。

PIC は、順序非依存の統合、idempotent な Canon、commitment boundary、安全三点の保守的集約を与える。V6 は自己進化そのものを定義しないが、自己進化が破壊的矛盾へ落ちないための前進的基盤を与える。

したがって、V7 Final では、Intelligence を単なる能力ではなく、PIC 上で安定的に自己更新できる構造として扱う。

F. Capability Gain と Self-Evolution の分離

能力向上と自己進化は同じではない。

本稿では次のように区別する。

能力向上は、現在の天井へ近づくことである。

自己進化は、天井そのものを更新することである。

英語では次の通りである。

Capability gain approaches the current ceiling. Self-evolution updates the ceiling itself.

モデルがより多くの問題を解けるようになること。より速く出力できること。より長い文脈を扱えること。より正確に検索できること。より高度なコードを書けること。これらは能力向上である。

しかし、自己進化とは、単に現在の性能限界へ近づくことではない。

失敗、矛盾、再構成誤差、外部アンカーからの反論、記憶圧縮、完了失敗などを通して、未来の限界そのものを変えることである。

ただし、V7 Final ではこの点を強く言いすぎない。自己進化の完全定義を与えるのではなく、guardable self-recursive evolution の許容条件として扱う。

安全な表現は次である。

能力向上は自己進化の必要条件になり得るが、それだけでは十分条件ではない。

能力が向上しても、方向がなければ空転する。Guard がなければ危険になる。PIC がなければ破断する。External Anchor がなければ自己正当化閉鎖へ落ちる。V11/V12 の再接続条件がなければ、記憶と完了が未来自己へ渡らない。

したがって、能力向上は自己進化の素材にはなり得る。しかし、それ単独では自己進化ではない。

G. External Anchor の定義

自己回帰は、閉じたままでは危険である。

システムが自分の出力を評価し、自分の更新を正当化し、自分の記憶を圧縮し、自分の完了を判定するなら、自己正当化閉鎖へ落ちる可能性がある。

この失敗を避けるために、V7 Final は External Anchor を要求する。

本稿における External Anchor の定義は次である。

外部アンカーとは、自己回帰が自己正当化閉鎖へ崩れることを防ぐ、十分に独立した矛盾・評価・連続性保持制約の源である。

英語では次のように表せる。

An external anchor is any sufficiently independent source of contradiction, evaluation, or continuity-preserving constraint that prevents self-recursive collapse into self-justifying closure.

External Anchor の条件は、主体ではなく機能で定義される。必要なのは次の性質である。

- 十分に独立していること
- 改ざん耐性を持つこと
- 矛盾源であること
- 評価源であること
- 連続性保持制約を与えること
- 自己回帰システムの内部都合だけで消せないこと

したがって、External Anchor は必ずしも人間だけを意味しない。ただし、現在の実装形では、人間による決定権が最も重要な外部アンカーである。

H. Human Ownership の位置づけ

V7 Final は、AI に主体性を与えない。

自己回帰システムを運用構造として扱い、決定権と責任は外部アンカー側に残す。

この関係を、次のように定義する。

人間による決定権は、現在の外部アンカーの実装形であって、唯一の形とは限らない。

英語では次の通りである。

Human ownership is the present implementation of external anchoring, not the only possible form.

この文により、二つの誤読を避ける。

第一に、V7 Final は人間を永久に唯一のアンカーとして固定するものではない。将来、十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与えられる別の構造が成立する可能性は、論理的には残る。

第二に、V7 Final は現在の AI に完全自律的な自己進化権限を与えるものではない。現時点では、人間の Seat、責任、承認、公開判断、停止判断が外部アンカーとして必要である。

つまり、本稿の立場は次である。

人間必須を形而上学的に主張しない。

しかし、現在の実装では人間の決定権を外さない。

このバランスにより、V5 の human final consent と、将来の非人間アンカー可能性を両立させる。

I. External Anchor が防ぐ失敗

External Anchor がない自己回帰は、内部整合性だけで閉じる。その場合、次の失敗が起きやすい。

第一に、自己正当化閉鎖である。システムは自分の出力を自分の前提にし、自分に都合のよい説明で閉じる。

第二に、後知恵書き換えである。過去の As-of 条件が消え、未来の能力や新しい情報から、過去判断が都合よく再解釈される。

第三に、記憶圧縮の自己汚染である。システムが自分の要約を次の根拠にし続け、元の証拠、停止条件、未知、制約が薄くなる。

第四に、False Completion である。現在自己には完了に見えるが、未来自己が再開できない形で閉じる。

第五に、Guard 回避である。自己回帰が自分の更新を正当化する中で、Guard 条件を「不要」「例外」「すでに満たした」と扱う。

External Anchor は、これらを完全に消すものではない。しかし、内部閉鎖に対して外部から矛盾、評価、停止、再接続条件を与える。

したがって、External Anchor は、自己回帰の敵ではない。むしろ、自己回帰を進化として扱うための防御的前提である。

J. $A \times G \times I$ の最小解釈

ここまでをまとめると、V7 Final の最小解釈は次の通りである。

この四つは、V7 Final の最小防御構造である。

繰り返すが、これは AGI 完成条件ではない。自己進化の発生を保証する条件でもない。

これは、自己回帰的進化を主張する前に、少なくとも落とすといけない構造的失敗を避けるための条件である。

V7 Final が問うのは、「AGI とは何か」を一挙に答えることではない。問うのは、次である。

自己回帰的 AGI 定義は、Aspire、Guard、PIC 上の Self-Recursion、External Anchor を欠いたまま、なぜ失敗しないと言えるのか。

もしその問いに答えられないなら、その定義は V7 Final の関所を通過していない。

TABLE XIII
MINIMAL INTERPRETATION OF $A \times G \times I$ AND EXTERNAL ANCHOR.

要素	役割
A: Aspire	自己回帰が空転しないための方向条件
G: Guard	自己回帰が不可逆害、制御不能化、Carrier 破壊へ落ちないための境界条件
I: Intelligence on PIC	自己回帰的更新が破断、順序依存、安全希釈へ落ちないための統合条件
External Anchor	自己回帰が自己正当化閉鎖へ落ちないための外部矛盾・評価・連続性保持条件

K. 次章への接続

本章では、 $A \times G \times I$ と External Anchor を定義した。

次章では、これらを具体的な Failure Mode Coverage Map へ展開する。

C1–C9 は、自己回帰的進化を発生させるチェックリストではない。各条件を欠いたときに、どの失敗モードが再開するかを示す地図である。

したがって、次章の中心文は次である。

これらを満たしたから自己進化が始まるとは主張しない。主張するのは、満たさない系が既知の失敗モードへ繰り返し崩壊するというだけである。

XVI. FAILURE MODE COVERAGE MAP C1–C9

C1–C9 は、自己進化を発生させるためのチェックリストではない。また、「これを満たせば AGI である」と主張する十分条件でもない。

本章で提示する C1–C9 は、Failure Mode Coverage Map である。すなわち、各条件を外すと、どの既知の失敗モードが再開するかを示す地図である。

中心文は次である。

これらを満たしたから自己進化が始まるとは主張しない。主張するのは、満たさない系が既知の失敗モードへ繰り返し崩壊するというだけである。

この章の目的は、自己回帰的進化を証明することではない。むしろ、自己回帰的進化を主張するシステムが、どの失敗群を最低限避けなければならないかを明示することである。

A. C1 Direction / Aspire

C1 は、方向条件である。

自己回帰システムは、更新するだけでは足りない。何に向かって更新するのが必要である。

Aspire が欠けると、自己回帰は空の一貫性ループへ落ちる。システムは整合的に見える出力を作り続けるが、構造差分を生まない。短期評価や局所的な滑らかさに最適化され、長期方向を失う。

欠落時の失敗モードは次である。

- 空の一貫性ループ
- 方向なき自己回帰
- 短期最適化
- vacuous recursion

したがって、C1 は「良い目標」を持つ条件ではない。自己回帰が空転しないための、最小の方向条件である。

B. C2 Guard

C2 は、境界条件である。

自己回帰システムは、内部更新や外部実行を行う。そのため、どこまで更新してよく、どこから先は止めるべきかを定義しなければならない。

Guard が欠けると、更新は危険な attractor へ入り得る。能力が増えるほど、危険な実行、不可逆害、自己保存的な閉鎖、Carrier 破壊が強化される可能性がある。

欠落時の失敗モードは次である。

- unsafe attractor
- irreversible harm
- unguardable update
- 実行境界の消失

Guard は、知能の後に追加される付属品ではない。自己回帰を進化として許容するための構成条件である。

C. C3 PIC / Canonical Integration

C3 は、非破壊的統合条件である。

自己回帰システムは、複数の出力、記憶、証拠、反論、更新候補を統合する。このとき、統合規則がなければ、順序依存、last-write-wins、過度な圧縮、安全シグナルの希釈が起きる。

PIC が欠けると、更新は統合ではなく破断になる。

欠落時の失敗モードは次である。

- rupture
- order-dependent merge
- last-write-wins
- 非再現的統合
- commitment 破壊

V6 の PIC は、自己進化そのものではない。しかし、自己進化が破壊的矛盾へ落ちないための前進化的基盤である。

D. C4 Self-Recursion

C4 は、自己回帰条件である。

外部から調整されるだけのシステムは、自己回帰的進化ではない。一回だけ改善されるシステムも、自己回帰的進化ではない。

Self-Recursion とは、システムが自分自身の構造を再利用し、更新し、再評価することである。ただし、それは PIC 上で安定化されなければならない。

C4 が欠けると、システムは自己回帰ではなく、外部チューニングや単発の適応に留まる。

欠落時の失敗モードは次である。

- 外部チューニング
- 一回きりの改善
- 自己回帰ではなく適応に留まる
- self-modification without stable recursion

したがって、C4 は「変われること」ではない。変化を自己の更新構造へ再利用できることを要求する。

E. C5 Trajectory

C5 は、軌道条件である。

自己回帰的進化は、点では評価できない。ある更新が局所的に有効でも、長期軌道としては漂流や破断を生むことがある。

Trajectory が欠けると、評価は点評価になる。短期性能、短期整合性、短期報酬、短期ベンチマークだけで進化を判断してしまう。

欠落時の失敗モードは次である。

- 点評価だけになる
- drift を見落とす
- 軌道損傷
- branch collapse

C5 は、変化を時間の線として見るための条件である。進化は、単発の性能向上ではなく、未来の分岐と Returnability を含む軌道として評価されなければならない。

F. C6 As-of Responsibility

C6 は、As-of 責任条件である。

自己回帰システムは、未来の状態から過去を再評価する。これは有用である一方で、危険でもある。未来の能力や情報を使って、過去判断を後から都合よく書き換えることができるからである。

As-of Responsibility が欠けると、過去判断の合理性が消える。当時の情報、制約、道具、能力上限、未知が失われ、後知恵の説明だけが残る。

欠落時の失敗モードは次である。

- hindsight rewrite
- 後からもっともらしい説明で過去判断が消える
- 当時合理性の消去
- responsibility evaporation

重要なのは次である。

AI は間違ふことで人間を傷つけるだけではない。後からもっともらしい説明を作り、当時の判断の合理性を消すことでも人間を傷つける。

したがって、自己回帰には、過去を後知恵から守る構造が必要である。

G. C7 Survival-Bounded Planning

C7 は、Carrier 保存条件である。

Aspire は強い方向を与える。しかし、Aspire の追求が Carrier を壊すなら、その軌道は継続できない。

Carrier とは、Aspire を運ぶ人間、組織、関係、資源、時間、信用、回復力である。Goal がどれほど重要でも、Carrier が壊れれば、未来軌道は失われる。

C7 が欠けると、目標が生存基盤を消費する。夢が運び手を壊す。成功が自己破壊になる。

欠落時の失敗モードは次である。

- 目標が生存基盤を消費する
- 夢が運び手を壊す
- 成功が自己破壊になる
- carrier collapse

V10 の原則は、V7 Final にもそのまま接続する。

Aspire must not consume or outrun the carrier of its own trajectory.

日本語では次のように言える。

Aspire は、自らの軌道を運ぶ Carrier を消費し尽くしてはならない。

H. C8 Reconnectable Forgetting

C8 は、再接続可能な忘却条件である。

自己回帰システムは、すべてを完全に記憶することはできない。しかし、単純な削除や要約では、未来自己が過去判断へ戻れなくなる。

Reconnectable Forgetting が欠けると、記憶は二つの失敗へ落ちる。一つは memory obesity である。すべてを抱えすぎて、現在判断が重くなる。もう一つは discontinuity である。忘却や圧縮によって、証拠、As-of、停止条件、未解決差分、再入経路が消える。

欠落時の失敗モードは次である。

- memory obesity
- discontinuity
- forgetting breaks the line
- 前提腐食
- single-context recursion による自己参照腐食

V11 の原則は、次の一文に圧縮できる。

Future requires forgetting, but forgetting must remain reconnectable.

日本語では次の通りである。

未来には忘却が必要である。しかし、忘却は再接続可能でなければならない。

I. C9 Completion Integrity

C9 は、未来再開可能な完了条件である。

自己回帰システムにおいて、完了は単に作業を終えることではない。未来自己が再開、検証、停止、再アンカーできる形で閉じることである。

Completion Integrity が欠けると、False Completion が起きる。現在自己には完了に見えるが、未来自己には再開できない。未解決の負担、証拠欠落、停止条件欠落、軌道漂流が、完了したふりをして未来へ渡される。

欠落時の失敗モードは次である。

- false completion
- future self が再開できない
- hidden burden
- control-loss handoff

V12 の中心文は次である。

Completion is not proven correctness. Completion is future-restartable closure.

日本語では次の通りである。

完了とは、正しさが証明された状態ではない。未来自己が再開可能な形で閉じられた状態である。

J. Coverage Map としての C1-C9

C1-C9 は、足し算ではない。また、万能な安全保証でもない。

これらは、自己回帰的進化を主張するシステムが、少なくとも無視できない失敗地形を覆うための Coverage Map である。

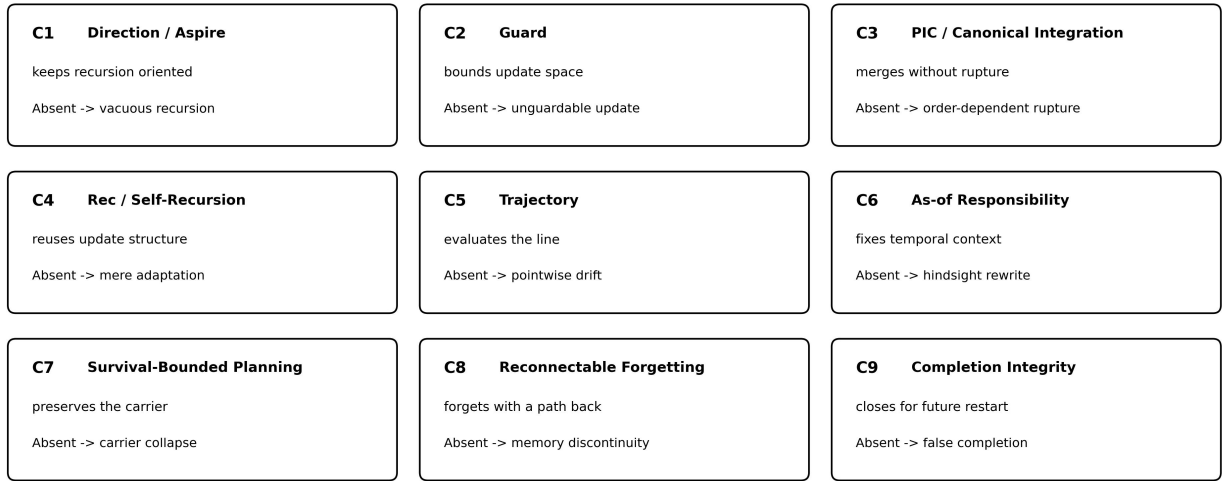
この表は、十分条件の表ではない。自己回帰的進化を主張するシステムが、少なくともどの失敗群を回避、反証、または置換しなければならないかを示す表である。

TABLE XIV
FAILURE MODE COVERAGE MAP C1–C9.

条件	役割	欠けた場合
C1 Direction / Aspire	方向	空転、短期最適化、vacuous recursion
C2 Guard	境界	unsafe attractor、不可逆害、unguardable update
C3 PIC / Canonical Integration	非破壊の統合	rupture、順序依存、last-write-wins
C4 Self-Recursion	自己回帰	外部チューニング、単発適応
C5 Trajectory	軌道評価	点評価、drift 見落とし、branch collapse
C6 As-of Responsibility	時点責任	後知恵書き換え、当時合理性の消去
C7 Survival-Bounded Planning	Carrier 保存	Carrier collapse、成功による自己破壊
C8 Reconnectable Forgetting	再接続可能な記憶	memory obesity、記憶断絶、前提腐食
C9 Completion Integrity	未来再開可能な完了	False Completion、control-loss handoff

Figure 2. Failure Mode Coverage Map (C1–C9)

C1–C9 are not sufficient conditions; each marks a failure mode that reopens when the condition is absent.



Coverage claim: systems that fail these conditions repeatedly collapse into known failure modes.

The map defines a defensible research target, not an AGI completion checklist.

Fig. 5. Failure Mode Coverage Map for V7 Final. C1–C9 do not define sufficient conditions for AGI; they show which failure modes reopen when each condition is missing.

K. C1–C9の同時性

C1–C9は、独立したチェック項目として足し合わせるものではない。自己回帰的進化において、これらは連鎖する。方向がなければ空転する。Guardがなければ危険領域へ漂流する。PICがなければ更新は破断する。Self-Recursionがなければ進化は外部チューニングに留まる。

Trajectoryがなければ点評価へ戻る。As-of責任がなければ過去が後知恵で書き換わる。Carrier保存がなければ軌道を運ぶ主体が壊れる。再接続可能な忘却がなければ記憶が腐食する。Completion Integrityがなければ未来自己が壊れた前提から再開する。

したがって、C1–C9は次のように読むべきである。

C1–C9は、自己進化を発生させる条件ではない。自己回帰的進化を主張する前に、失敗として再開してはならない地形の一覧である。

L. 次章への接続

本章では、C1–C9をFailure Mode Coverage Mapとして定義した。

次章では、これらの条件を単なるリストとしてではなく、Precondition Topologyとして再配置する。すなわち、どの条件が入口であり、どの条件が境界であり、どの条件が記憶・軌道・完了の維持条件なのかを整理する。

この再配置により、V7 FinalはC1–C9を「AGI判定チェックリスト」としてではなく、自己回帰的進化の失敗地形を読むための構造地図として扱う。

TABLE XV
OUTER FAILURE REGIONS IN THE PRECONDITION TOPOLOGY.

外側失敗領域	対応する欠落
vacuous recursion	Direction / Aspire の欠落
unguardable update	Guard の欠落
rupture	PIC / Canonical Integration の欠落
mere adaptation	Self-Recursion の欠落
trajectory damage	Trajectory の欠落
hindsight rewrite	As-of Responsibility の欠落
carrier collapse	Survival-Bounded Planning の欠落
memory discontinuity	Reconnectable Forgetting の欠落
false completion	Completion Integrity の欠落
self-justifying closure	External Anchor の欠落

XVII. PRECONDITION TOPOLOGY

V7 Final は、AGI そのものを定義する論文ではない。また、自己進化を発生させる十分条件を証明する論文でもない。本章で定義するのは、Precondition Topology である。すなわち、guardable self-recursive evolution が安全に始まるための前提地形である。

自己回帰的進化を主張するシステムは、単に「自分を更新できる」と言うだけでは足りない。その更新がどの失敗地形を避け、どの境界内に留まり、どの条件を満たしたときに初めて研究対象として妥当になるのかを示さなければならない。

V7 Final が描くのは、その地形である。

A. Precondition Topology とは何か

Precondition Topology とは、ある主張が成立する前に、どの前提条件が満たされていないかを示す構造地図である。

V7 Final の場合、対象となる主張は次である。

自己回帰的知能は、更新を続けながら、守れる形で進化できる。

この主張を立てるには、単に能力が高いことを示すだけでは足りない。自己更新能力を示すだけでも足りない。ベンチマーク上の汎用性を示すだけでも足りない。

必要なのは、自己回帰が次の失敗へ落ちないことを示す前提地形である。

- 空転
- 漂流
- 破断
- 自己正当化閉鎖
- 後知恵書き換え
- 記憶断絶
- Carrier collapse
- False Completion

Precondition Topology は、これらの失敗領域を外側に置き、その内側に guardable self-recursive basin を置く。つまり、V7 Final の地形は次のように読める。

- 外側には、既知の失敗領域がある。
- 中央には、守れる自己回帰的進化の盆地がある。
- 入口には、 $A \times G \times I$ がある。
- その内側に入るには、C1-C9 が Failure Mode Coverage として機能する必要がある。

これは AGI 完成の地図ではない。研究対象として安全に扱える自己回帰的進化の入口地図である。

B. 外側失敗領域

Precondition Topology の外側には、自己回帰的進化を破壊する既知の失敗領域がある。

これらの失敗領域は、単独で現れるとは限らない。むしろ、自己回帰的システムでは連鎖する。

方向がない自己回帰は空転する。Guard がない更新は危険領域へ漂流する。PIC がない統合は破断する。軌道評価がなければ点評価へ戻る。As-of 責任がなければ過去が後知恵で書き換わる。Carrier 保存がなければ軌道の運び手が壊れる。記憶が再接続できなければ前提が腐食する。完了が未来再開可能でなければ、次の自己は壊れた前提から始まる。

したがって、Precondition Topology は、失敗を単発のエラーとして扱わない。自己回帰が長期運用で落ちる地形として扱う。

C. 中央盆地：Guardable Self-Recursive Basin

Precondition Topology の中央には、Guardable Self-Recursive Basin がある。

これは、AGI 完成領域ではない。能力が自由に拡大する領域でもない。自己回帰的進化を研究対象として扱う前に、最低限そこへ入っていないと防衛的な盆地である。

この盆地に入るには、少なくとも次の条件が必要である。

- 1) Aspire によって方向が残ること
- 2) Guard によって境界が残ること
- 3) PIC によって更新と統合が破断しないこと
- 4) External Anchor によって自己正当化閉鎖へ沈まないこと
- 5) C1-C9 によって既知の失敗地形が再開しないこと

この盆地は、完全な安全を保証しない。自己進化の発生も保証しない。保証するのは、より狭いことだけである。

この範囲に入らない自己回帰的進化の主張は、既知の失敗地形を未処理のまま残している。

V7 Final は、その未処理部分を可視化する。

D. 図式：Precondition Topology

本稿の Figure 3 は、次の構造を想定する。

- 中央に、Guardable Self-Recursive Basin を置く。
- 入口に、 $A \times G \times I$ を置く。
- 中央を囲む形で、C1-C9 の Coverage 条件を置く。
- 外側に、失敗領域を置く。

外側の失敗領域は次の通りである。

- vacuous recursion
- unguardable update
- rupture
- mere adaptation
- trajectory damage
- hindsight rewrite
- carrier collapse
- memory discontinuity
- false completion
- self-justifying closure

図の意味は、AGI の完成領域を示すことではない。自己回帰的進化を安全に研究対象へ乗せる前に、どの失敗地形を解決すべきかを示すことである。正本でも、この図は中央盆地・ $A \times G \times I$ 入口・C1-C9 coverage・外側失敗領域の構成として説明されている。

キャプション候補は次の通りである。

Guard 可能な自己回帰的進化の前提地形。本図は AGI そのものを定義しない。外部アンカー付き自己回帰が安全に不要となる前に解決されなければならない構造的失敗群を示す。

英語では次のように置ける。

Precondition topology for guardable self-recursive evolution. The figure does not define AGI itself. It defines the class of structural failures that must be resolved before externally anchored recursion can become safely unnecessary.

E. カオスの漂流と Attractor Basin

自己回帰的更新では、小さな差分が長期的に大きな軌道差へ増幅され得る。

この意味で、V7 Final の Failure Mode は、カオスの漂流と近い構造を持つ。ただし、本稿はカオス理論を主役にしない。ここで必要なのは、厳密な力学系モデルではなく、次の直観である。

小さな更新差分は、安定化構造がなければ、長期軌道の大きな分岐へつながる。

そのため、自己回帰には、attractor-like な canonical structure が必要になる。

V7 Final では、この役割を次の三要素が担う。

- Aspire は方向を与える。
- Guard は許容領域を境界づける。
- PIC は安定化する canonical substrate として機能する。

したがって、V7 Final の Basin は、能力が自由に拡大する領域ではない。方向、境界、統合規則によって、更新が守れる範囲へ戻れる領域である。

Figure 3. Precondition Topology

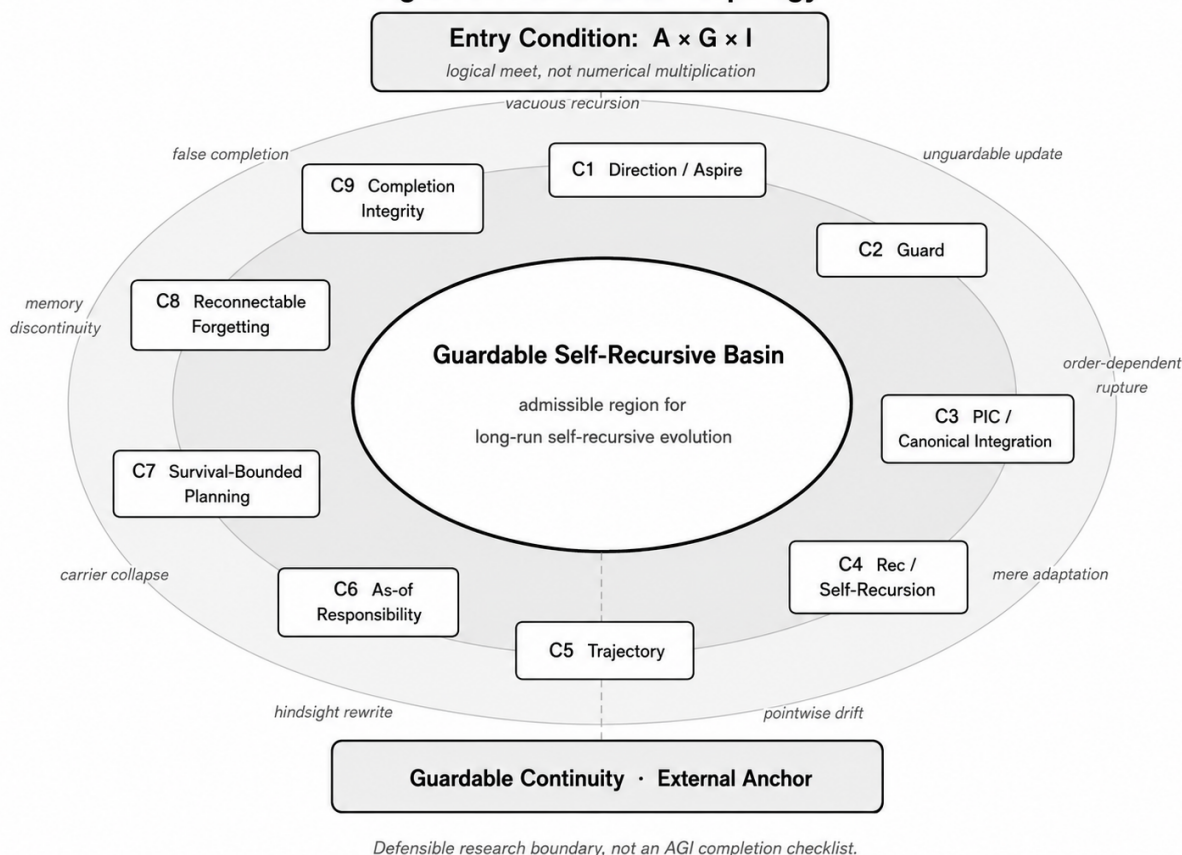


Fig. 6. Precondition topology for guardable self-recursive evolution. The figure does not define AGI itself; it shows the structural failure regions that must be resolved before externally anchored recursion can become safely unnecessary.

F. External Anchor 付き自己回帰がなぜ必要か

Precondition Topology の重要点は、External Anchor を中央の外側に置くことではない。External Anchor は、中央盆地の成立条件である。

自己回帰システムが自分の出力を評価し、自分の更新を正当化し、自分の記憶を圧縮し、自分の完了を判定するなら、そのシステムは内部整合性だけで閉じる危険がある。

このとき、外部からの矛盾、評価、停止、再接続条件がなければ、自己回帰は自己正当化閉鎖へ落ちる。

External Anchor は、この内部閉鎖を破るために必要である。

ただし、External Anchor は必ずしも人間だけを意味しない。必要なのは、十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与え、自己回帰システムの内部都合だけで消せないことである。

現在の実装形では、人間の Seat、承認、責任、公開判断、停止判断が最も重要な External Anchor である。しかし、V7 Final は人間を形而上学的に唯一のアンカーとして固定しない。

本稿の主張は、より限定的である。

外部アンカーなしの自己回帰を主張するなら、外部アンカーが果たしていた矛盾・評価・停止・再接続機能を、どの構造が置換するのかを示さなければならない。

これを示せない自己回帰は、Precondition Topology の中央盆地には入らない。

G. Precondition Topology の最小主張

本章の最小主張は次である。

V7 Final は、AGI の完成領域を描かない。自己回帰的進化を安全に研究対象へ乗せる前に、どの失敗地形を解決・反証・置換しなければならないかを描く。

この意味で、Precondition Topology は判定表ではない。ランキングでもない。AGI 認定の基準でもない。

それは、自己回帰的進化を主張する前に、少なくとも避けなければならない失敗地形の配置図である。

この配置により、V7 Final は「AGI とは何か」を過大に宣言しない。むしろ、問いを反転させる。

自己回帰的進化を主張するシステムは、なぜ空転せず、漂流せず、破断せず、自己正当化せず、過去を書き換えず、記憶を断絶せず、Carrier を壊さず、False Completion へ落ちないと言えるのか。

この問いに答えられないなら、その定義はまだ V7 Final の前提地形を通過していない。

H. 次章への接続

本章では、C1–C9 を Precondition Topology として再配置した。

次章では、この地形の中で、失敗や再構成誤差をどのように観測可能な更新材料へ変換するかを扱う。

中心となる概念は、Reconstructive Delta である。

ただし、Reconstructive Delta は、進化そのものの証明ではない。差分が出たから進化した、という主張でもない。

それは、再構成誤差を検出し、停止し、構造特定し、未来の自己回帰更新へ渡すための観測可能な運用ハンドルである。

XVIII. OPERATIONALIZATION: RECONSTRUCTIVE DELTA

ここまでで、V7 Final は理論的な前提地形を定義した。

しかし、自己回帰的進化を運用上扱うには、もう一つの問いが残る。

失敗、欠落、忘却、圧縮、再構成誤差は、どのようにして自己回帰更新の材料になり得るのか。

本章では、そのための最小概念として Reconstructive Delta を定義する。

ただし、ここで強く主張しすぎてはならない。

Reconstructive Delta は、自己進化の完全なメカニズムではない。また、これを観測すれば自己進化が発生したと証明できる、というものでもない。

本章の主張は限定的である。

V7 Final は、再構成誤差を単なる失敗としてではなく、自己回帰更新材料になり得る観測可能な差分として扱う。

英語では次のように表せる。

V7 treats reconstruction error not as failure, but as possible evolutionary material.

A. なぜ Reconstructive Delta が必要か

自己回帰的システムは、過去の状態をそのまま保持し続けるわけではない。

記憶は圧縮される。文脈は切れる。要約が作られる。別のモデルが引き継ぐ。未来自己が過去を読み直す。欠落した構造を補完する。

このとき、必ず再構成が起きる。

再構成には誤差がある。

普通なら、この誤差は失敗として扱われる。元の構造と違う。抜けがある。解釈がずれた。補完が過剰だった。前提が変わった。

しかし、V7 Final では、この誤差を一段違う角度から見る。

再構成誤差は、単なるノイズとは限らない。それは、現在のシステムが過去の構造をどう読み、どこを補い、何を失い、何を新しく接続したかを示す差分でもある。

この差分が、自己回帰更新材料になり得る。

ただし、差分が出たこと自体を進化とは呼ばない。

重要なのは、差分が検出され、停止可能であり、構造として特定され、未来の更新ルールへ渡せることである。

B. Reconstructive Delta の定義

本稿における Reconstructive Delta を次のように定義する。

Reconstructive Delta とは、未来自己が過去の構造を再構成したときに、元の構造との間に生じる、検出可能で、構造特定可能で、未来更新へ送れる差分である。

より短く言えば、次の通りである。

Reconstructive Delta は、再構成誤差を未来更新のための構造差分として扱う運用ハンドルである。

ここで重要なのは、三つの限定である。

第一に、Reconstructive Delta は誤差そのものではない。誤差が観測され、比較され、構造として特定されたときに、初めて Delta になる。

第二に、Reconstructive Delta は進化そのものではない。Delta は進化材料になり得るが、Delta の発生だけでは進化を証明しない。

第三に、Reconstructive Delta は Guard の外へ出てはならない。差分が強くても、Guard、PIC、External Anchor、Completion Integrity を壊すなら、その差分は進化材料ではなく危険な漂流である。

TABLE XVI
THREE EXPERIMENTAL HANDLES FOR RECONSTRUCTIVE DELTA.

ハンドル	内容	目的
Aspire Delta	人間の Aspire と AI 出力のズレ	方向が保存されているかを見る
Missing-Link Reconstruction Delta	欠落した中間構造を AI がどう補完するか	欠落補完の癖と誤差を見る
Reconnective Forgetting Delta	忘却・圧縮後に AI が何を再構成するか	再接続可能性と前提腐食を見る

C. Error から Delta への変換

V12 では、エラーが自己進化の材料になるには、Detect、Stop、Identify、Delta の変換を通る必要がある。すなわち、エラーは検出され、停止可能になり、構造原因として特定され、未来の更新ルールへ送られて初めて進化材料になり得る。

V7 Final の Reconstructive Delta も、この考えに従う。

最小経路は次である。

Reconstruction Error → Detect → Stop → Identify → Delta → Future Update

この流れを本文上では次のように読む。

- 再構成誤差がある。
- それを検出する。
- 危険なら止める。
- 何がずれたのかを構造として特定する。
- その差分を未来更新へ渡す。

この変換がない場合、誤差は単なる誤差に留まる。

Detect がなければ、誤差は見えない前提になる。

Stop がなければ、誤差は次の更新へ拡大する。

Identify がなければ、同じ損傷が繰り返される。

Delta がなければ、未来更新の材料にならない。

したがって、Reconstructive Delta は、失敗を美化する概念ではない。失敗を未来更新へ渡せる形に変換するための運用条件である。

D. 三つの実験的ハンドル

Reconstructive Delta は、本文の理論核ではなく、V7 Final から導かれる実験的ハンドルとして扱う。

正本末尾の追補では、本文に大きく入れすぎると「V7 はこれらを検証済みの進化機構として主張している」と誤読される危険があるため、本文では短く扱い、詳細は Appendix または Addendum 候補とする方針が示されている。

本稿では、最小限、次の三つを置く。

これらは、自己回帰システムが、欠落・不整合・忘却された構造をどのように再構成するかを調べるための実験的ハンドルである。

繰り返すが、これらは完全な進化メカニズムの主張ではない。

E. Aspire Delta

Aspire Delta は、人間の Aspire と AI 出力のズレである。

自己回帰的システムが、過去の方角、目的、制約、文脈を引き継いだつもりでも、出力が微妙に別方向へ寄ることがある。

このズレは、単なる品質低下とは限らない。それは、AI がどの方向を Aspire として読んだのか、どの制約を重く見たのか、どの未来方向を補完したのかを示す観測材料になる。

ただし、Aspire Delta は、常に良い差分ではない。

AI 出力が人間の Aspire から離れた場合、それは方向更新ではなく、Aspire Drift である可能性がある。したがって、Aspire Delta は必ず Guard と External Anchor の下で評価される。

安全な表現は次である。

Aspire Delta は、方向の進化を証明するものではない。人間の Aspire と AI の再構成方向のズレを観測するためのハンドルである。

F. Missing-Link Reconstruction Delta

Missing-Link Reconstruction Delta は、欠落した中間構造を AI がどのように補完するかを見るためのハンドルである。最小例は次である。

元の構造が、1-2-3-4-5 である。

そこから 2 を抜く。

AI には 1-3-4-5 だけを渡す。

AI が欠落した 2 をどう補完するかを見る。

最後に、AI が補完した 2 と、元の 2 を比較する。

この比較で見えるのは、AI が単に正解を当てたかどうかではない。

- AI が欠落をどう読んだか。
- どの関係を重視したか。
- どの構造を自然な接続だと判断したか。
- どこで過剰補完したか。
- どこで元の文脈を失ったか。

これらが、Missing-Link Reconstruction Delta になる。

このハンドルは、自己回帰的システムが欠落を補完するときの癖を観測するために有用である。ただし、補完が元構造に近いことをもって自己進化とは言わない。補完がズレたことをもって進化とも言わない。

重要なのは、そのズレが検出され、構造特定され、未来の再構成ルールへ送れるかである。

G. Reconnective Forgetting Delta

Reconnective Forgetting Delta は、忘却・圧縮後の再構成差分である。

自己回帰的システムは、すべてを完全には覚えられない。そのため、記憶は圧縮され、文脈は短くなり、未来自己は圧縮残渣から過去を再構成する。

このとき、すぐに原構造へ戻さず、いったん AI に再構成させる。

その後、再構成された構造と原構造を比較する。

そこで生じる差分が、Reconnective Forgetting Delta である。

この Delta が示すのは、圧縮後にも何が残ったか、何が消えたか、何が誤って補完されたか、どの停止条件や証拠アンカーが薄くなったかである。

V11 の言葉で言えば、これは再接続可能な忘却が成立しているかを見るための観測ハンドルである。

ただし、忘却後に別の構造が出たからといって、それを進化とは呼ばない。

忘却によるズレは、前提腐食かもしれない。False Completion の入口かもしれない。あるいは、未来自己が失われた構造を新しい接続で回復した可能性もある。

それを判別するには、Evidence Anchor、As-of 条件、Stop Condition、External Anchor が必要である。

H. Reconstructive Delta と Guard

Reconstructive Delta は、Guard の外では危険である。

なぜなら、再構成差分は魅力的に見えるからである。AI が欠落を補い、新しい構造を作り、過去より滑らかな説明を出すと、それは進化に見える。

しかし、滑らかさは進化ではない。整合性も進化ではない。差分の発生も進化ではない。

V7 Final では、Reconstructive Delta が進化材料になり得るのは、少なくとも次の条件を満たす場合に限られる。

- Aspire と接続している
- Guard を越えていない
- PIC 上で破断せず統合できる
- External Anchor によって自己正当化閉鎖を避けられる
- Evidence Anchor へ戻れる
- Stop Condition が残っている
- Future Restartability を壊さない

この条件を満たさない Reconstructive Delta は、進化材料ではなく、漂流材料である。

I. 差分の定着論

追補 B の論点は、Reconstructive Delta を扱ううえで重要である。

差分は、出た瞬間に価値を持つとは限らない。差分が一回現れただけでは、それはノイズかもしれない。偶然の補完かもしれない。文脈破損かもしれない。

したがって、V7 Final では「差分が出た＝進化」とは扱わない。

差分が自己回帰更新材料になるには、少なくとも次の条件が必要である。

- 1) 差分が検出されている
- 2) 元構造との比較が可能である
- 3) 差分が Aspire、Guard、PIC、External Anchor の下で評価されている
- 4) 差分が未来更新へ渡せる形で記録されている
- 5) 次回以降の再構成や判断で、同じ失敗を減らす、または有効な構造差分を増やす方向に働く

このとき初めて、差分は単なるズレではなく、更新材料として定着し始める。

つまり、差分の定着とは、差分が一回現れることではない。差分が未来の自己回帰に再利用可能な形で残ることである。

J. Reconstructive Delta の射程と限界

Reconstructive Delta は強い概念である。しかし、その射程は限定される。

Reconstructive Delta は、自己進化の証明ではない。AGI 判定基準でもない。能力向上指標でもない。完全な実験プロトコルでもない。

それは、自己回帰的システムが過去構造を再構成するとき、どの差分が生じ、その差分を未来更新へ送れるかを見るための運用ハンドルである。

本稿の立場は次である。

Reconstructive Delta は、V7 Final の理論核ではなく、V7 Final から導かれる観測可能な実験的ハンドルである。

したがって、本文では最小限に留める。詳細なプロトコル、評価基準、実験設計、定量化は、Appendix または Addendum、あるいは後続研究へ送る。

この限定により、V7 Final は主張過多を避ける。

K. 次章への接続

本章では、Reconstructive Delta を定義した。

再構成誤差は、単なる失敗として閉じられるだけではない。それは、検出され、停止可能であり、構造として特定され、未来更新へ送れる場合、自己回帰更新材料になり得る。

ただし、Reconstructive Delta は進化そのものではない。

次章では、V7 Final の Scope Boundary を固定する。すなわち、本稿が何を主張し、何を主張しないのかを明確にする。特に重要なのは、次の境界である。

- V7 Final は、AGI そのものを定義しない。
- 自己進化の発生を証明しない。
- C1-C9 を十分条件として扱わない。
- Reconstructive Delta を進化証明として扱わない。
- External Anchor を人間必須論へ縮退しない。
- Capability growth を authority expansion として扱わない。

XIX. SCOPE BOUNDARY / LIMITATIONS / FAQ

本章では、V7 Final の射程を固定する。

ここまでの章で、V7 Final は Decision-OS を Gate-first operational architecture として定義し、Guardable Continuity Axiom、 $A \times G \times I$ 、External Anchor、C1-C9、Precondition Topology、Reconstructive Delta を提示した。

しかし、これらは AGI 完成宣言ではない。自己進化の発生証明でもない。自己回帰的進化を主張する前に、どの構造的失敗群を処理しなければならないかを示すための防御的正準形である。

したがって、本章では、本稿が何を主張し、何を主張しないのかを明確にする。

A. 本稿が主張すること

V7 Final が主張するのは、次である。

第一に、Decision-OS は、判断・更新・統合・記憶・公開・完了をまたいで、守れる連続性を保存するための Gate-first operational architecture である。

第二に、自己回帰的進化は、能力向上だけでは定義できない。変化が Guardable Continuity を保存しないなら、その変化は V7 Final の意味では進化ではない。

第三に、 $A \times G \times I$ は、数値積ではなく logical meet である。Aspire、Guard、PIC 上の Self-Recursion は、自己回帰的進化を研究対象として扱うために同時に必要な入口条件である。

第四に、External Anchor は、自己回帰が自己正当化閉鎖へ落ちないために必要な、十分に独立した矛盾・評価・連続性保持制約の源である。

第五に、C1-C9 は、自己進化を発生させるチェックリストではなく、各条件を欠いたときに再開する失敗モードを示す Failure Mode Coverage Map である。

第六に、Reconstructive Delta は、再構成誤差を自己回帰更新材料へ変換し得る観測可能な運用ハンドルである。ただし、差分の発生そのものを進化とは呼ばない。

B. 本稿が主張しないこと

V7 Final は、次を主張しない。

- AGI そのものを完全定義したとは主張しない。
- 意識を定義しない。
- 主観的自己を定義しない。
- 自己進化を発生させる十分条件を証明しない。
- $A \times G \times I$ を満たせば AGI が完成すると主張しない。

- C1-C9 を AGI 成立チェックリストとして扱わない。
- Reconstructive Delta を自己進化の証明として扱わない。
- External Anchor を人間必須論へ縮退しない。
- 能力向上を権限拡大として扱わない。
- Decision-OS の構成要素を無制限に他領域へ拡大解釈してよいとは主張しない。

本稿の目的は、AGI の全体像を閉じることではない。自己回帰を含む AGI 定義が、どの構造的失敗を無視してはならないかを定義することである。

C. 存在論から認識論へ

V7 Final の重要な境界は、存在論から認識論への移動である。

本稿は、「このシステムは AGI である」と存在論的に断定するものではない。

むしろ、問いを次の形へ移す。

自己回帰的進化を主張するシステムは、どの条件を満たせば、少なくとも既知の失敗地形を未処理のまま残していないと言えるのか。

これは、存在の断定ではなく、検査可能性の設計である。

V7 Final は、AGI の本質を完全に言い当てることを目指さない。代わりに、自己回帰的進化を主張する系が通過しなければならぬ構造的失敗境界を示す。

したがって、本稿の強さは「AGI を定義した」という宣言にはない。強さは、自己回帰的 AGI 定義が避けて通れない失敗群を、検査可能な形で並べたことにある。

D. Capability growth does not expand authority

モデル能力の上昇は、Decision-OS の解釈力を高める可能性がある。

より強いモデルは、Decision-OS をより深く読めるかもしれない。より多くの関係を見つけるかもしれない。より広い領域へ応用可能性を見出すかもしれない。

しかし、能力の上昇は権限の拡大を意味しない。

Capability growth does not expand authority.

より強いモデルが Decision-OS をより深く読めるとしても、それは Scope、Owner、Evidence Boundary、Gate を勝手に広げる権利を与えない。

Decision-OS の構成要素は、領域横断的に再利用され得る。しかし、その再利用は、宣言された Gate、Owner、Evidence Boundary を超える射程拡大を許可しない。

この境界は、未来の高性能 LLM が Decision-OS を勝手に拡大解釈するリスクを防ぐために必要である。正本追補でも、この項目は Scope Boundary / Limitations / Future-proofing paragraph へ置くべきものとして扱われている。

E. External Anchor の限界

External Anchor は重要である。しかし、それは完全な安全装置ではない。

External Anchor は、自己回帰が自己正当化閉鎖へ落ちることを防ぐための外部矛盾・評価・連続性保持制約である。だが、外部アンカーがあるだけで、すべての失敗が消えるわけではない。

外部アンカーが弱ければ、自己回帰システムはそれを形式的に通過するだけになる。外部アンカーが改ざん可能なら、内部閉鎖の一部になる。外部アンカーが責任を持たないなら、停止条件として機能しない。

したがって、External Anchor は「外に何かがあること」ではなく、次の機能を満たす必要がある。

- 十分に独立していること
- 改ざん耐性があること
- 矛盾を出せること
- 評価を返せること
- 停止・再接続・再アンカー条件を与えられること
- 自己回帰システムの内部都合だけで消せないこと

現時点では、人間の Seat、承認、責任、公開判断、停止判断が最も重要な実装形である。しかし、V7 Final は人間を形而上学的に唯一のアンカーとして固定しない。

F. Reconstructive Delta の限界

Reconstructive Delta も、限界を持つ。

再構成誤差が観測されたとしても、それは自己進化の証明ではない。欠落を補完できたとしても、それは AGI 判定ではない。忘却後に新しい構造が出たとしても、それが有効な進化材料とは限らない。

Reconstructive Delta が有効な更新材料になるには、少なくとも次の条件が必要である。

- 元構造との比較が可能である

- 差分が検出されている
- 危険な差分を止められる
- 差分の構造原因が特定されている
- Guard 境界内で扱われている
- PIC 上で統合可能である
- External Anchor によって検証可能である
- Future Restartability を壊さない

これらを満たさない差分は、進化材料ではなく、漂流材料である。

したがって、Reconstructive Delta は本文の理論核ではなく、V7 Final から導かれる実験的ハンドルとして扱う。

進化とは圧力そのものではない。圧力が再利用可能な残差を残し、それが伝達され、守られ、自己回帰的に改良される
とき、進化が始まる。初期人類における道具使用も、単なる能力向上ではなく、環境圧力に対する再利用可能な差分の定
着として読むことができる。V7 Final においても、Reconstructive Delta は失敗や誤差を美化する概念ではない。それは、
壊れずに残った差分を検出し、Guard 内で保持し、未来の自己回帰更新へ送るための運用ハンドルである。

自己進化は、最初から完成した因果線として観測されるとは限らない。実務上、それは複数の反応群を最小説明する因
果仮説への大域的圧縮として現れる場合もあれば、既存説明に回収できない局所的残差として現れ、後から構造的意味を
持つ場合もある。前者は検証と説明に強く、後者は早期発見に強い。どちらの場合も、外部アンカーなしでは、自己回帰
は自己正当化へ崩れる危険を持つ。

G. Scope Boundary

本稿の Scope Boundary は次の通りである。

V7 Final は AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群を定義する。

V7 Final は、自己進化を発生させる論文ではない。自己回帰的進化を主張する系が、どの失敗群を解決・反証・
置換しなければならないかを示す論文である。

V7 Final は、哲学の集大成ではない。Gate-first operational architecture としての Decision-OS が、guardable
continuity を自己回帰へ拡張した防御的正準形である。

この三文が、本稿の最終的な境界である。

H. FAQ

Q1. V7 Final は AGI を定義しているのか。

完全定義ではない。V7 Final は、AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群
を定義する。

Q2. $A \times G \times I$ を満たせば AGI なのか。

本稿はそう主張しない。 $A \times G \times I$ は、数値式でも完成条件でもない。それは、guardable self-recursive evolution を
研究対象として扱うための入口条件である。

Q3. C1–C9 はチェックリストなのか。

違う。C1–C9 は Failure Mode Coverage Map である。満たせば自己進化が起きる条件ではなく、欠けると既知の失敗
モードが再開する条件である。

Q4. External Anchor は人間必須論なのか。

違う。External Anchor は主体ではなく機能で定義される。現在の実装形では人間の Seat と責任が重要だが、論理的
には十分に独立し、改ざん耐性を持ち、矛盾・評価・連続性保持制約を与えられる別構造もあり得る。

Q5. Reconstructive Delta が出れば進化したと言えるのか。

言えない。差分は進化そのものではない。差分が検出され、停止可能であり、構造特定され、Guard 境界内で未来更新
へ送れる場合に限り、進化材料になり得る。

Q6. 能力が高いモデルなら Decision-OS の権限も広がるのか。

広がらない。Capability growth does not expand authority. より強いモデルがより深く読めても、Scope、Owner、
Evidence Boundary、Gate を勝手に広げる権利はない。

I. 次章への接続

ここまでで、V7 Final の本文主張は閉じた。

残るのは Conclusion である。

Conclusion では、新しい概念を追加しない。本稿が何を定義したのか、何を定義しなかったのか、そしてなぜ V7 Final
が自己回帰的 AGI 定義の構造的関所なのかを、最後に相応しい形で閉じる。

XX. CONCLUSION

本稿は、AGI そのものを完全定義するものではない。また、自己進化を発生させる十分条件を証明するものでもない。

V7 Final が定義したのは、自己回帰を含む知能定義が無視できない構造的失敗群である。自己回帰する知能が、更新を続けながらも、制御不能化、漂流、破断、自己正当化閉鎖、記憶断絶、Carrier collapse、False Completion へ落ちないためには、どの構造的境界を通過しなければならないのか。本稿は、その問いに対する防御的正準形を提示した。

中心にあるのは、Guardable Continuity Axiom である。

進化は、変化が守れる連続性を保存するときのみ許容される。

この一文により、V7 Final は、能力向上と自己進化を分離した。能力向上は、現在の天井へ近づくことである。自己進化は、天井そのものを更新することである。しかし、その更新が方向、境界、責任、記憶、再開可能性を壊すなら、それは進化ではなく、守れない変化である。

V7 Final の Door Formula は、次の通りである。

$$A \times G \times I = V4 \times V5 \times V6 = V7. \quad (12)$$

ただし、この「 $\times$ 」は数値積ではない。A、G、I を増やせば AGI 度が高まるという意味でもない。それは、構造的同時成立、すなわち logical meet である。

Aspire は方向を与える。

Guard は境界を与える。

PIC 上の Self-Recursion は、非破壊的な自己更新の基盤を与える。

この三つのいずれかが欠けると、自己回帰は空転、漂流、破断、または外部チューニングへ戻る。

さらに本稿は、External Anchor を、自己回帰が自己正当化閉鎖へ落ちないための外部矛盾・評価・連続性保持制約として定義した。External Anchor は人間必須論ではない。現在の実装形では人間の Seat、承認、責任、停止判断が最も重要であるが、定義上は主体ではなく機能で決まる。

C1-C9 は、自己進化を発生させる条件ではない。これらは、Failure Mode Coverage Map である。すなわち、各条件を外したときに、どの既知の失敗モードが再開するかを示す地図である。

Precondition Topology は、この Coverage Map を地形として再配置した。外側には、空転、漂流、破断、自己正当化閉鎖、後知恵書き換え、記憶断絶、Carrier collapse、False Completion がある。中央には、Guardable Self-Recursive Basin がある。V7 Final が描いたのは、AGI 完成領域ではなく、自己回帰的進化を研究対象として安全に扱うための入口地形である。

Reconstructive Delta は、その地形の中で、再構成誤差を自己回帰更新材料へ変換し得る観測可能な運用ハンドルである。ただし、Reconstructive Delta は進化そのものではない。差分が検出され、停止可能であり、構造として特定され、Guard 境界内で未来更新へ送れる場合に限り、それは進化材料になり得る。

したがって、V7 Final の最終主張は、強いが限定的である。

V7 Final は AGI そのものを定義しない。自己回帰を含む AGI 定義が無視できない構造的失敗群を定義する。

この制約は、AGI 研究を狭めるためではない。むしろ、自己回帰的進化を安全に研究対象へ乗せるためである。

未来の自己回帰システムは、V7 Final の語彙を使わないかもしれない。Decision-OS を引用しないかもしれない。A $\times$ G $\times$ I という式を採用しないかもしれない。

しかし、そのようなシステムであっても、自己回帰的進化を主張するなら、同じ失敗群に答えなければならない。

- なぜ空転しないのか。
- なぜ Guard を越えないのか。
- なぜ破断しないのか。
- なぜ自己正当化閉鎖へ落ちないのか。
- なぜ過去を後知恵で書き換えないのか。
- なぜ記憶を断絶しないのか。
- なぜ Carrier を壊さないのか。
- なぜ False Completion へ落ちないのか。

この問いに答えられない自己回帰的 AGI 定義は、まだ守れる自己回帰的進化の入口に立っていない。

V7 Final は、AGI の到達宣言ではない。自己回帰的 AGI 定義が進化可能性を主張する前に通過すべき構造的関所である。

DISCLOSURE AND USE OF AI TOOLS

This paper was developed by the author with the assistance of AI tools for drafting, restructuring, translation support, terminology refinement, and consistency checking.

All conceptual claims, framework decisions, final wording choices, publication decisions, and responsibility for the content remain with the author.

AI tools were used as assistants for iteration and editorial support. They were not treated as decision owners, empirical validators, or independent authorities. No AI system is credited as an author of this paper.

The framework proposed in this paper is conceptual and operational. It does not claim empirical validation, deployment testing, or implementation as a complete memory architecture. The proposed gates, layers, and criteria should be read as design primitives for long-horizon agentic AI systems rather than as experimentally verified components.

The author also notes that this paper is part of the broader Decision-OS research sequence. Earlier Decision-OS works introduced related concepts such as evidence anchoring, As-of fixation, impact-weighted release, survival-bounded planning, and completion integrity. V11 focuses specifically on reconnectable forgetting as a memory-economy condition for future re-entry.

REFERENCES

- [1] R. Xu, Z. Wang, R.-Z. Fan, and P. Liu, “Benchmarking benchmark leakage in large language models,” *arXiv preprint arXiv:2404.18824*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.18824>
- [2] J. Haimes, C. Wenner, K. Thaman, V. Tashev, C. Neo, E. Kran, and J. Schreiber, “Benchmark inflation: Revealing llm performance gaps using retro-holdouts,” *arXiv preprint arXiv:2410.09247*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.09247>
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Stabilization, Safety, and Security of Distributed Systems*, 2011.
- [4] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [5] L. Moreau and P. Missier, “Prov-dm: The prov data model,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [6] B. Fischhoff, “Hindsight is not equal to foresight: The effect of outcome knowledge on judgment under uncertainty,” *Journal of Experimental Psychology: Human Perception and Performance*, vol. 1, no. 3, pp. 288–299, 1975.
- [7] National Institute of Standards and Technology, “Artificial intelligence risk management framework (ai rmf 1.0),” U.S. Department of Commerce, National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [8] S. Nagata, “Decision-os v4: Polaris-origin,” 2026, decision-OS research sequence.
- [9] —, “Decision-os v5 revised (siriusa2): A zero-knowledge confirmation layer for trajectory-aware human-ai decision safety,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19828435>
- [10] —, “Decision-os v6: Canonical commitment architecture for non-destructive integration,” 2026, decision-OS research sequence.
- [11] —, “Decision-os v7 addendum: Why aspire and why pic,” 2026, decision-OS research sequence.
- [12] —, “Decision-os v8: Time-tube control for self-safe agi,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19690553>
- [13] —, “Decision-os v9.1: Impact-weighted release — from continuous-pass gates to condition-bound judgment reuse,” 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19935535>
- [14] —, “Decision-os v10: Survival-bounded planning for aspire-preserving ai assistance,” 2026, decision-OS research sequence.
- [15] —, “Decision-os v11: Forget for future — reconnectable forgetting for long-horizon agentic ai,” 2026, decision-OS research sequence.
- [16] —, “Decision-os v12: Completion integrity,” 2026, decision-OS research sequence.